

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ПРОЕКТИРАНЕ И РАЗРАБОТВАНЕ НА
Compile-time Object-Relational Mapping
библиотека на C++**

ДИПЛОМНА РАБОТА

Изготвил:

Алекс Цветанов, фак. № 121222225

Специалност: Компютърно и софтуерно инженерство

Научен ръководител:

доц. д-р инж. Галя Павлова

София, 2026

СЪДЪРЖАНИЕ

I.	Въведение	3
1.1.	Постановка на проблема	3
1.2.	Цел и задачи на разработката	3
1.3.	Обхват и ограничения	4
1.4.	Структура на изложението	5
II.	Преглед на съществуващите решения	6
2.1.	Класификация на подходите	6
2.2.	Критерии за сравнение	8
2.3.	Native клиентски библиотеки	9
2.4.	Runtime query builder-и	9
2.5.	Типизирани SQL DSL решения	10
2.6.	Класически ORM рамки и кодогенерация	10
2.7.	Граници на релационно центрираните абстракции	11
2.8.	Позициониране на настоящата разработка	12
2.9.	Изводи от сравнителния анализ	12
III.	Теоретични основи на Compile-time Object-Relational Mapping и моделите за съхранение	13
3.1.	Таксономия на problem domain-a	13
3.2.	Compile-time Object-Relational Mapping като архитектурен подход	14
3.3.	Статичен модел на данните в C++	15
3.4.	Типизиран query IR	17
3.5.	Теория на връзките и стратегии за съхранение	19
3.6.	Релационен модел	20
3.7.	Документен модел	20
3.8.	Key-value модел	20
3.9.	Графов модел	21
3.10.	Wide-column модел	21
3.11.	Тестова опора на теоретичните твърдения	22
3.12.	Граници на унифицираната абстракция	23

IV. Архитектура на библиотеката	24
4.1. Архитектурни цели и ограничения	24
4.2. Слоеста организация и разделяне на отговорностите	25
4.3. Entity слой като архитектурна основа	26
4.4. Query IR като централен архитектурен компонент	27
4.5. Конекторен слой и формалният contract	27
4.6. Потребителският фасаден слой db<DB>	29
4.7. Миграции като архитектурно разширение	30
4.8. Архитектурни инварианти	30
4.9. Архитектурен извод	31
V. Реализация на основните подсистеми	32
5.1. Repository-backed mini case study: User и Post	32
5.2. Скалярни свойства и именуване на контейнерите	33
5.3. Field wrappers, правила и placeholder-и	33
5.4. Изграждане на query обекти	34
5.5. Изпълнение, резултати и достъп до полета	35
5.6. Prepared queries като повторно използваем execution артефакт	36
5.7. Миграционен слой и DDL генериране	37
5.8. Нишкова безопасност и transaction helpers	38
5.9. Извод за реализацията	39
VI. Сценарии за работа с релационни и нерелационни бази данни	41
6.1. Методика на анализа	41
6.2. Релационен базов сценарий: SQLite	42
6.3. Релационна преносимост: PostgreSQL и MySQL	43
6.4. Документен сценарий: MongoDB	43
6.5. Key-value сценарий: Redis	44
6.6. Графов сценарий: Neo4j	44
6.7. Wide-column сценарий: Cassandra	45
6.8. Съпоставка между test evidence и backend contract	47
6.9. Сравнителен синтез	48
VII. Верификация и надграждаемост чрез нови конектори	49

7.1. Формален contract на конекторния слой	49
7.2. Задължителни елементи на минимален конектор	50
7.3. Типизиране, signatures и именуване	51
7.4. Capability механизъм и compile-time ограничения	52
7.5. Минимален connector skeleton и operational connector skeleton	52
7.6. Транзакции, нишкова безопасност и prepared queries	54
7.7. Миграции и schema-aware интеграция	54
7.8. Методика за добавяне на нов backend	55
7.9. Критерии за „напълно имплементиран и функциониращ“ конектор	56
7.10. Верификация чрез тестове	57
7.11. Оценка на наличните конектори	57
VIII. Ограничения и бъдещо развитие	58
8.1. Стратегически направления за развитие	58
8.2. Разширяване на live backend покритието	59
8.3. Пълна schema-aware интеграция и runtime introspection	60
8.4. Нишкова безопасност и жизнен цикъл на връзките	61
8.5. Ниско ниво на изпълнение и wire protocol оптимизации	61
8.6. Интеграция с C++26 reflection и допълнителна автоматизация	62
8.7. Разширяване на connector contract-а и capability таксономията	62
8.8. Необходимост от по-строга емпирична оценка	63
8.9. Обобщение	64
IX. Заключение	65
9.1. Обобщение на решената задача	65
9.2. Обобщение на постигнатите резултати	66
9.3. Научни и приложни приноси	67
9.4. Основни ограничения и граници на обобщението	68
9.5. Значение за практиката и за бъдещите изследвания	69
9.6. Заключителни бележки	69
X. Речник на използваните съкращения и понятия	70
10.1. Основни съкращения	70
10.2. Работни дефиниции на ключови понятия	71

10.3. Нотация и именуване в текста	72
10.4. Репозиториен контекст на най-често цитираните артефакти	73
10.5. Извод	74
XI. Карта на верификационните сценарии и репозиторийните артефакти . .	75
11.1. Обща структура на доказателствения слой	75
11.2. Каталог на основните unit тестове	76
11.3. Integration и live тестове	78
11.4. Sarability доказателства по backend семейства	79
11.5. CMake registration като част от verification дизайна	80
11.6. Изводи от приложението	81
XII. Атлас на sarability профилите и зрелостта на backend-ите	83
12.1. Sarability механизмът като формален слой	83
12.2. Зрелост на конектора като стълбица, а не като двоична оценка	84
12.3. Sarability профили по backend семейства	85
12.4. Карта на зрелостта по налична тестова опора	87
12.5. Какво показва sarability atlas-ът за архитектурата	89
12.6. Извод	89

РЕЗЮМЕ

Настоящата дипломна работа разглежда проектирането, реализацията и оценяването на Compile-time Object-Relational Mapping библиотека за C++23 [6]. Основната идея на разработката е, че логическата структура на заявките и на модела на данните не трябва да се описва чрез низове, интерпретирани на етап изпълнение, а чрез типизирани `constexpr` обекти, валидирани от компилатора. По този начин част от традиционните грешки при достъпа до данни — неправилни имена на полета, несъвместими типове, липсващи части от заявката и погрешно използване на неподдържани операции — се преместват от фазата на изпълнение към фазата на компилация.

Разработената библиотека използва статичен модел на данните, изразен в C++ чрез конструкции като `property<T,Name>`, `relationship<...>`, `table_name_trait<T>` и типизиран `query IR`, представен чрез `select_query`, `insert_query`, `update_query` и `delete_query`. Този междинен модел е независим от конкретния тип база данни. Материализацията към релационни и нерелационни системи се осъществява чрез специализации на `connector_trait<DB>`, които декларират поддържаните възможности и превеждат една и съща логическа заявка към SQL, BSON-подобни филтри, Redis команди, Cypher или CQL в зависимост от избрания backend.

В работата се анализират както релационните сценарии за SQLite, PostgreSQL и MySQL, така и нерелационните сценарии за MongoDB, Redis, Neo4j и Cassandra. Показано е как един и същ C++ модел диктува логическата схема, но се материализира различно като таблица, документ, ключ-стойност структура, графов възел или wide-column ред. Особено внимание е отделено на ролята на `store_as::reference` и `store_as::embed` като обща абстракция за връзки между обекти, която се интерпретира според модела на съхранение.

Верификацията на разработката се основава на unit и integration тестове от хранилището. Те покриват изграждането на query IR, capability-базираното ограничаване на операции, подготвените заявки, миграциите, нишковата безопасност и поведението на конкретните конектори при работа с реални или mock драйвери. Това позволява формулиране не само на архитектурен модел, но и на конкретни критерии за „напълно имплементиран и функциониращ“ конектор.

Основният принос на дипломната работа е в три направления. Първо, предложен е практически приложим модел за Compile-time Object-Relational Mapping библиотека, който съчетава статична типова безопасност и backend-независима архитектура. Второ, показано е как C++ описанието на данните може да се използва като първичен източник за логическа схема при различни типове бази данни. Трето, формализиран е connector contract, чрез който библиотеката може да се надгражда с нови backend реализации без промяна в user-facing query API.

Ключови думи: C++23, Compile-time Object-Relational Mapping, query IR, connector_trait, типова безопасност, релационни и нерелационни бази данни, подготвени заявки, миграции

I. ВЪВЕДЕНИЕ

1.1. Постановка на проблема

Достъпът до данни остава едно от местата, в които разминаването между езика на приложението и модела на съхранение създава най-много структурни грешки. В традиционните слоеве за достъп до бази данни заявките се описват като низове, които се валидират едва при изпълнение. Това означава, че неправилни имена на колони, несъвместими типове, невалидни JOIN конструкции или неправилно подадени параметри се откриват късно — често върху реална база данни и след допълнителни ръчни тестове. Същевременно смяната на backend от релационен към документен, графов или key-value модел обикновено изисква пълно пренаписване на слоя за достъп до данни.

Обектно-релационното картографиране е класическият подход за свързване на обектния модел на приложението с персистентното съхранение [2]. Повечето ORM реализации обаче остават силно зависими от runtime механизми: SQL низове, кодогенерация, анотации, макроси или динамични описания на схема. В контекста на C++ това създава допълнителен проблем. Езикът предлага мощна система от шаблони, constexpr изчисления и concepts, но тези механизми често не се използват докрай в библиотеките за достъп до данни. Настоящата работа изследва доколко е възможно компилаторът да поеме ролята на първи валидатор на query структурата и на backend-съвместимостта.

Съществена особеност на разработваната система е, че въпреки името Compile-time Object-Relational Mapping библиотека, архитектурата ѝ не е ограничена само до релационния модел. Напротив, тя използва еднакъв C++ модел на данните и еднакъв query IR като логически слой над релационни и нерелационни системи. Това поставя втори изследователски въпрос: кои части на ORM абстракцията са универсални и кои неизбежно трябва да се ограничават чрез capability-базиран backend contract.

1.2. Цел и задачи на разработката

Основната цел на дипломната работа е да представи проектирането и реализацията на Compile-time Object-Relational Mapping библиотека за C++, която:

1. моделира данните и заявките чрез статично типизирани C++ конструкции;
2. позволява изграждане на SELECT, INSERT, UPDATE и DELETE заявки като constexpr

обекти;

3. отделя логическата структура на заявката от backend-специфичното ѝ материализиране;
4. поддържа релационни и нерелационни конектори чрез `connector_trait<DB>`;
5. използва `compile-time capability` механизъм за ограничаване на неподдържани операции;
6. демонстрира как C++ моделът определя логическата схема и физическата структура на данните при различни backend-и;
7. формализира `contract` за добавяне на нови конектори;
8. валидира архитектурата чрез систематични `unit` и `integration` тестове.

За постигане на тази цел работата решава следните задачи: анализ на съществуващи решения; дефиниране на статичен `entity` модел; изграждане на `query IR`; проектиране на `connector layer` и `capability tags`; имплементиране на конектори за различни типове бази данни; изследване на миграции, подготвени заявки и нишкова безопасност; формулиране на критерии за разширяемост.

1.3. Обхват и ограничения

Работата разглежда библиотеката като слой за моделиране, изграждане и изпълнение на заявки. В обхвата ѝ попадат `entity` моделът, `query IR`, `placeholder` механизмът, подготвените заявки, миграционният модел, `connector trait` архитектурата, `capability` ограничаването и конкретните backend реализации в хранилището. Съществена част от оценяването е сценарийният анализ на SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j и Cassandra.

Извън непосредствения обхват остават производствено ниво оптимизации като разпределено управление на връзки, автоматизирани `migration` стратегии за всички backend-и, цялостен `benchmarking` върху големи натоварвания и пълна експлоатация на бъдещата `reflection` функционалност на C++26. Тези направления са разгледани като перспектива в Глава VIII.

1.4. Структура на изложението

Дипломната работа е организирана по следния начин.

Глава II разглежда съществуващи C++ решения за достъп до бази данни и ORM подходи и позиционира разработката спрямо тях.

Глава III представя теоретичните основи на Compile-time Object-Relational Mapping и моделите за съхранение, необходими за по-късния анализ на релационни и нерелационни сценарии.

Глава IV описва архитектурата на библиотеката: entity описанията, query IR, connector_trait, capability механизмите и мястото на миграциите в общия модел.

Глава V проследява реализацията на основните подсистеми и показва как те се свързват на ниво код и runtime поведение.

Глава VI анализира как една и съща логическа абстракция се материализира в различни backend-и — релационни, документни, key-value, графови и wide-column.

Глава VII разглежда тестовата верификация и формалния contract за надграждане чрез нови конектори.

Глава VIII очертава ограниченията на текущата версия и възможните направления за бъдещо развитие, а **Глава IX** обобщава резултатите и приносите.

II. ПРЕГЛЕД НА СЪЩЕСТВУВАЩИТЕ РЕШЕНИЯ

Анализът на съществуващите решения е необходим, за да се оцени дали предложената архитектура действително допринася с нова стойност, или само преформулира вече познати практики. При C++ библиотеките за достъп до данни могат да се разграничат две независими оси: нивото на абстракция и моментът, в който се изгражда и валидира заявката. Нивото на абстракция варира от почти директно използване на клиентски C API до пълно ORM поведение. Моментът на валидиране може да бъде runtime, hybrid или compile-time.

Тези две оси са особено важни в контекста на настоящата дипломна работа. Ако се наблюдава само “удобството на API-то”, лесно може да се пропусне съществената разлика между библиотека, която просто прави SQL низовете по-четими, и библиотека, която действително прехвърля структурната коректност към компилатора. Ако, от друга страна, се гледа само кога възниква грешката, може да се подцени цената на различните нива на абстракция, build toolchain сложност и пригодност за работа с повече от един storage модел.

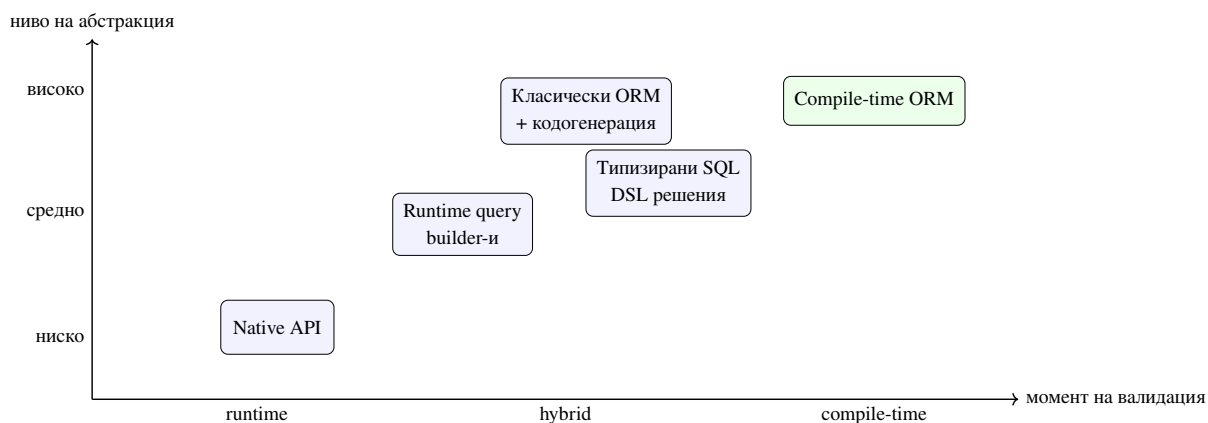
2.1. Класификация на подходите

В най-ниския слой стоят native клиентските библиотеки като SQLiteC API [4], libpq [8] и MySQLConnector/C [7]. Те предоставят максимален контрол върху заявките, параметрите и резултатите, но не предлагат логическа абстракция. Приложението е принудено да работи директно със SQL низове, индекси на колони, позиции на параметри и backend-специфични типове.

Следващата група са runtime query builder библиотеките. Те обикновено подобряват четимостта на заявките и намаляват част от шаблонния код, но продължават да разчитат на runtime изграждане на текстово представяне. Типовата безопасност е частична и обикновено спира до границата на API повикването. Структурната валидност на заявката остава проблем на изпълнението.

Най-високото ниво на абстракция е ORM подходът. Там C++ класовете се асоциират с таблици, колони и връзки, а библиотеката генерира необходимите заявки. Предимството е намаляване на boilerplate кода. Недостатъкът е, че често се въвеждат външни генератори, макроси, анотации и opaque runtime метаданни, които затрудняват

проследимостта между C++ кода и реалната заявка.



Фигура 1. Позициониране на основните семейства решения според абстракция и момент на валидация

Фигура 1 показва, че compile-time ORM подходът не е просто “още една ORM библиотека”. Той стои в горния десен квадрант, където високото ниво на абстракция се комбинира с ранна структурна проверка. Тази комбинация е рядка, защото изисква повече архитектурна дисциплина в самата библиотека.

Таблица 1. Съпоставка на основните подходи за достъп до данни в C++

Подход	Ниво на абстракция	Валидация	Предимство	Основен недостатък
Native API	ниско	runtime	пълнен контрол и близост до драйвера	силна обвързаност с конкретен backend
Runtime query builder	средно	основно runtime	по-четим API и по-малко boilerplate	заявката пак се материализира като низ
Типизиран SQL DSL	средно към високо	частично compile-time	по-добра статична структура за SQL света	обикновено остава ограничен до SQL
Класически ORM	високо	runtime или чрез кодогенерация	удобно object mapping поведение	сложна toolchain интеграция и непрозрачност
Compile-time ORM	високо	compile-time за структурата	ранно откриване на структурни грешки и формален connector contract	по-висока сложност на шаблонния слой

2.2. Критерии за сравнение

За да бъде сравнението академично пълно, не е достатъчно да се каже, че една библиотека е “по-удобна”, а друга е “по-ниско ниво”. В настоящата работа се използват пет сравнителни критерия:

1. **Ниво на логическа абстракция** — доколко библиотеката позволява моделът на данните да се описва независимо от непосредствения синтаксис на backend-а.
2. **Степен на статична валидация** — кои аспекти на заявката и на модела могат да бъдат проверени преди изпълнение.
3. **Преносимост между backend-и** — възможно ли е логическият модел да бъде адаптиран към повече от един тип storage система.

4. **Сложност на toolchain-a** — изисква ли библиотеката външна кодогенерация, допълнителни preprocess стъпки или генератори на metadata.
5. **Прозрачност на грешките и на изпълнението** — доколко разработчикът може да проследи връзката между C++ кода, генерираната заявка и runtime поведението.

Тези критерии са пряко свързани с целите на дипломната работа. Ако предложената библиотека трябва да бъде едновременно expressive и формално проверяема, тогава тя трябва да превъзхожда алтернативите не само по удобство, а по баланса между тези пет измерения.

2.3. Native клиентски библиотеки

SQLite [3], libpq и MySQLConnector/C са представителни за подхода, при който приложението конструира и подава заявките директно към драйвера. Това е най-универсалният и често най-бързият вариант, но поставя тежестта на correctness изцяло върху приложния код. Разработчикът трябва ръчно да поддържа синхрон между имената на полета, поредността на параметрите и реалната схема на базата данни. При промяна на backend-a се променя не само диалектът на заявката, но и целият API за връзка, подготвяне, bind и четене на резултати.

Този подход е особено проблематичен, когато един и същ домейн модел трябва да бъде свързан последователно с повече от един тип база данни. В такъв случай приложението започва да натрупва условни пътища, backend-специфични SQL низове и дублирана логика за hydration на резултати. След определен размер на системата това намалява не само удобството, но и проверимостта на коректността.

Силната страна на native API подхода е неговата честност. Той не обещава повече абстракция, отколкото реално предоставя. Именно затова той често служи като контролна точка, спрямо която могат да бъдат оценявани по-високите abstraction слоеве.

2.4. Runtime query builder-и

Библиотеки като SOCI [5] и част от по-лекия fluent SQL tooling търсят баланс между четимост и контрол. Те предлагат по-удобен C++ интерфейс за изграждане на заявки, но в много случаи продължават да разчитат на runtime описване и последващо рендиране към

SQL. В практиката това означава, че известна част от структурата на заявката е по-добре капсулирана, но backend-независимата смяна на модел за съхранение остава ограничена.

За настоящата разработка тези библиотеки са важни като отправна точка. Те показват, че fluent API и типизирани елементи са полезни, но не са достатъчни сами по себе си. Същинският въпрос е дали компилаторът може да валидира цялата логическа структура на заявката и дали същата абстракция може да бъде пренесена извън SQL света.

2.5. Типизирани SQL DSL решения

sqlpp11 [9] е особено интересен случай, защото стои между традиционния runtime query builder и по-строго статичния дизайн. Подобни библиотеки използват шаблони и типове, за да направят част от SQL структурата видима за компилатора. Това е съществена стъпка напред спрямо чисто runtime builders, защото част от структурните грешки могат да бъдат хванати по-рано.

Същевременно обаче типизираният SQL DSL обикновено остава дълбоко свързан с релационния модел. Дори когато е много силен като C++ API, той най-често предполага свят, в който таблици, колони, joins и агрегати остават централната метафора. Това прави такива решения изключително ценни за SQL домейна, но по-трудно преносими към document, graph или key-value системи.

Именно тук се очертава една от основните разлики с настоящата разработка. Тя не цели просто compile-time структуриране на SQL. Тя цели compile-time логически слой, който може да бъде материализиран различно според backend семейството.

2.6. Класически ORM рамки и кодогенерация

ODB [1] е характерен пример за ORM рамка, която предлага богато картографиране между класове и релационни таблици, но използва генерация на код и външен tooling. Това улеснява потребителя на ниво употреба, но измества сложността към build процеса и към автоматично генерираните артефакти. Получава се силна зависимост между модела на данните, допълнителната toolchain стъпка и конкретните релационни backend-и.

В класическите ORM рамки често се наблюдава и друго напрежение: колкото по-високо е удобството на ниво application код, толкова по-непрозрачен става пътят до реалната заявка и до performance характеристиките на системата. Това не е задължително фатален проблем, но в академичния контекст на настоящата работа е важно ограничение,

защото затруднява проследимостта между абстракцията и материализацията.

2.7. Граници на релационно центрираните абстракции

За приложения, които трябва да работят с документни, графови или key-value системи, класическият ORM модел показва естествени ограничения. Част от концепциите — като JOIN, външни ключове и строго таблична схема — нямат директен еквивалент във всички модели за съхранение. Поради това обобщението на ORM отвъд релационния свят не може да бъде постигнато само чрез генериране на SQL.

Това наблюдение е важно и за native API, и за SQL DSL решенията. Дори когато те са много силни технически, обикновено предполагат, че SQL е централният език на света на данните. Настоящата разработка приема по-различна отправна точка: логическият модел трябва да бъде първичен, а релационната материализация — само една от възможните интерпретации.

Таблица 2. Ограничения на различните семейства решения спрямо multi-backend целта

Семейство решения	Какво прави добре	Къде се появява ограничението
Native API	максимален контрол и близост до драйвера	липса на общ логически модел и голямо дублиране при повече от един backend
Runtime builders	по-добра четимост и капсулиране на заявката	валидацията остава предимно runtime и често е SQL-центрична
Типизирани SQL DSL	по-силна статична структура за SQL заявки	логиката обикновено остава ограничена в рамките на релационната метафора
Класически ORM	удобно object mapping поведение за релационни системи	зависимост от кодогенерация и трудна преносимост към нерелационни модели

2.8. Позициониране на настоящата разработка

Предложената библиотека се позиционира като Compile-time Object-Relational Mapping решение, което комбинира силна статична типизация, липса на външна кодогенерация и connector-базиран модел за пренасяне на логическата абстракция към различни backend-и. В сравнение с native API подхода тя намалява дублирането на логиката и риска от късно открити грешки. В сравнение с runtime query builder решенията премества структурната валидация към компилатора. В сравнение с класическите ORM рамки тя избягва външни генератори и поставя ясен акцент върху contract-a на конектора.

Ключовата разлика обаче е по-дълбока. Настоящата разработка не разглежда compile-time техниките като средство за “по-умен SQL builder”, а като начин за формализиране на логическия модел на достъпа до данни. Точно това позволява следващите глави да преминат от теорията на типизираните модели към архитектурата, connector contract-a и реалните multi-backend сценарии.

2.9. Изводи от сравнителния анализ

Сравнението със съществуващите решения води до три основни извода. Първо, native API подходът остава незаменим като baseline за контрол и performance, но не предлага удовлетворително ниво на логическа абстракция. Второ, runtime query builder-ите и типизираните SQL DSL решения подобряват работата със SQL, но не решават напълно проблема за backend-независим логически слой. Трето, класическите ORM рамки предлагат висока абстракция, но често за сметка на toolchain сложност и ограничена пригодност извън релационния свят.

Точно това позициониране оправдава по-детайлното разглеждане в следващата глава на два отделни, но свързани въпроса: кои езикови и теоретични механизми позволяват compile-time ORM подход и как един и същ логически модел може да бъде интерпретиран в релационни и нерелационни системи.

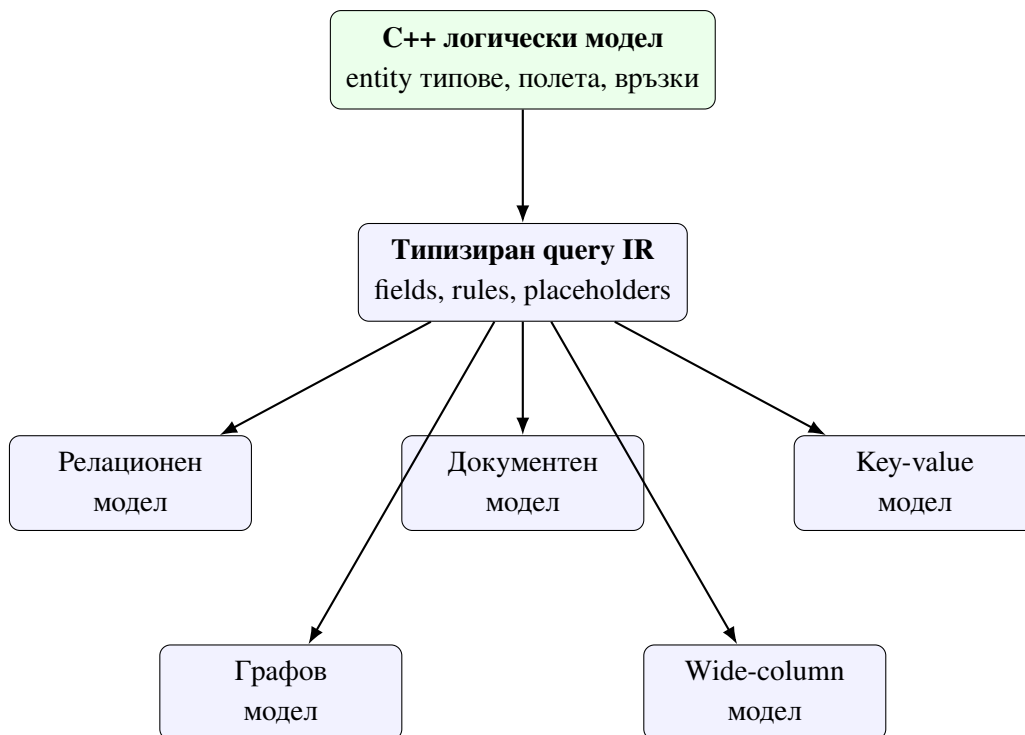
III. ТЕОРЕТИЧНИ ОСНОВИ НА COMPILE-TIME OBJECT-RELATIONAL MAPPING И МОДЕЛИТЕ ЗА СЪХРАНЕНИЕ

Теоретичната основа на настоящата разработка не се свежда до една езикова възможност или до един тип база данни. Библиотеката комбинира идеи от ORM теорията, статичното типизиране в C++, compile-time изграждането на междинни представяния и сравнителния анализ на различни модели за съхранение. За да бъде разбираема по-късната архитектура, е необходимо първо да се отдели логическият слой на описване на данните от физическия слой на тяхното backend-специфично материализиране.

В контекста на тази дипломна работа терминът Compile-time Object-Relational Mapping не трябва да се разбира стеснено само като SQL техника. По-точно той обозначава архитектурен подход, при който моделът на данните, формата на заявките и част от ограниченията върху операциите се описват чрез типовата система на езика, а не чрез runtime конфигурация. Това позволява една и съща логическа структура да бъде пренесена към релационни, документни, key-value, графови и wide-column системи, без backend-специфичните различия да се скриват или подменят.

3.1. Таксономия на problem domain-a

Преди разглеждане на отделните компоненти е полезно да се фиксира общата картина на problem domain-a. От една страна стои C++ моделът, в който приложението описва entity типове, полета, връзки и ограничения. От друга страна стоят различни семейства системи за съхранение, всяко със собствен модел на физическа организация, достъп и оптимизация. Между тях е разположен типизираният query IR, който служи като посредник между логиката на приложението и конкретния connector слой.



Фигура 2. Таксономия на логическия модел, query IR и backend семействата

Фигура 2 подчертава централното теоретично допускане на библиотеката: общото между backend-ите не е техният синтаксис, а логическият модел на данните и операциите. Следователно валидната абстракция трябва да бъде разположена по-високо от SQL, BSON, Redis команди, Cypher или CQL. Тя трябва да работи на равнището на полета, предикати, projection-и, връзки и допустими операции.

3.2. Compile-time Object-Relational Mapping като архитектурен подход

Класическото ORM свързва обектния модел на приложението с неговата персистентна схема [2]. В compile-time варианта това свързване се конструира не чрез runtime reflection и конфигурации, а чрез типове, шаблони и constexpr стойности. Практически това означава, че изборът на полета, допустимите операции и част от съвместимостта с backend-а се проверяват от компилатора. Грешки като обръщение към несъществуващо поле или използване на JOIN върху конектор без supports_joins престават да бъдат runtime изненади.

Съществената особеност е, че compile-time ORM не означава липса на runtime данни. Стойностите на параметрите се подават в момента на изпълнение, но структурата

на заявката е вече фиксирана и валидирана. Така се получава ясно разграничение между *логическа форма* и *runtime стойности*. Това разграничение е ключово за тезата, защото позволява едновременно статични гаранции и реална работа с външни бази данни.

От теоретична гледна точка този подход доближава ORM слоя до компилаторен pipeline. Приложният код описва семантична структура, compile-time компонентите я превръщат в междинно представяне, а едва най-долният слой материализира тази структура в backend-специфичен език или протокол. Затова по-късните архитектурни решения в библиотеката могат да се анализират през понятия като междинно представяне, статична проверка и backend materialization.

3.3. Статичен модел на данните в C++

В разглежданата библиотека домейн моделът се описва чрез C++ агрегати, чиито полета са от тип `property<T,Name>`. По този начин всяко поле носи едновременно стойностен тип и символно име на колоната или атрибута. Допълнително `table_name_trait<T>` позволява типът да бъде свързан с логически контейнер за съхранение — таблица, колекция, `key prefix`, `label` или друго backend-специфично понятие.

Тази организация има важно теоретично следствие: C++ типът става първичен носител на логическата схема. Отделните backend-и не налагат нов потребителски модел, а само интерпретират един и същ модел по различен начин. Именно затова по-късно може да се говори за еднакъв логически обект, който в релационен backend се превръща в таблица, а в документен — в документ с вложени или референтни полета.

Таблица 3. Съпоставка между C++ конструкциите и теоретичните ORM понятия

Конструкция	Теоретична роля	Практически ефект в библиотеката
<code>property<T,Name></code>	атрибут от логическата схема	носи тип на стойността и символно име за материализация
<code>relationship<...></code>	логическа връзка между обекти	отделя връзката от физическия механизъм за съхранение
<code>table_name_trait<T></code>	логически контейнер	служи като име на таблица, колекция, key prefix или label
<code>field<&T::m></code>	адресиране на атрибут в query пространство	свързва entity слоя с query IR-a
<code>connector_trait<DB></code>	backend материализация	определя как логическият модел се изпълнява физически

Следователно ORM слойът не започва от драйвер или от SQL рендериране. Той започва от статично описание на смисъла на данните. Това е и причината entity слойът да бъде фундаментален за цялата архитектура: ако този слой е неясен или непълен, няма как по-долу да се получи коректна и надграждаема материализация.

```

1  template <typename T, detail::string_literal Name>
2  struct property
3  {
4      using value_type = T;
5
6      [[nodiscard]] static constexpr std::string_view column_name() noexcept
7      {
8          return Name.view();
9      }
10
11     T value{};
12 };
13
14 enum class store_as { reference, embed };
15
16 template <store_as Strategy, typename T, detail::string_literal Name>
17 struct relationship
18 {
19     static constexpr store_as strategy = Strategy;
20     using related_type = T;
21
22     [[nodiscard]] static constexpr std::string_view field_name() noexcept
23     {
24         return Name.view();

```

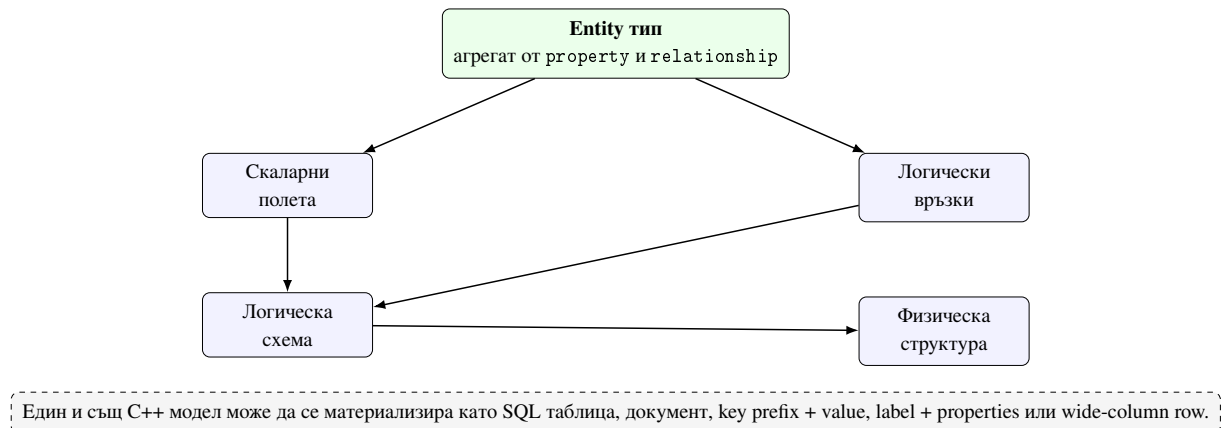
```

25     }
26 };

```

Listing 1: Извадка от `property` и `relationship` в `entity` слой

Listing 1 показва, че `entity` слойът е минималистичен по форма, но достатъчно богат по смисъл. `property` не е просто контейнер за стойност; той носи и име, а `relationship` не е просто `member field`; тя фиксира стратегията за материализация като част от типа.



Фигура 3. Преминаване от C++ `entity` описание към логическа схема и физическа материализация

3.4. Типизиран query IR

Вместо заявката да се описва директно като SQL или друго текстово представяне, библиотеката използва типизиран query IR. Неговите елементи са `field`, `Rule`, `Placeholder` и query обекти като `select_query`, `insert_query`, `update_query` и `delete_query`. Този модел дава две предимства. Първо, query структурата може да бъде анализирана на `compile-time` и да участва в `static_assert` проверки. Второ, един и същ IR може да бъде рендиран към различни backend езици и протоколи.

Теоретично IR-ът играе ролята на междинен език между домейн модела и конкретния слой за изпълнение. Това е аналогично на междинните представяния в компилаторите: потребителят работи с високо ниво на абстракция, а backend-ът получава специализирана форма, оптимизирана за собствения си модел. Именно поради това query IR-ът е логически по-близо до семантиката на операциите, отколкото до синтаксиса на конкретна заявка.

```

1 template <typename Response, typename Joins, typename Wheres,
2         typename Limits, typename Groups, typename Orders>

```

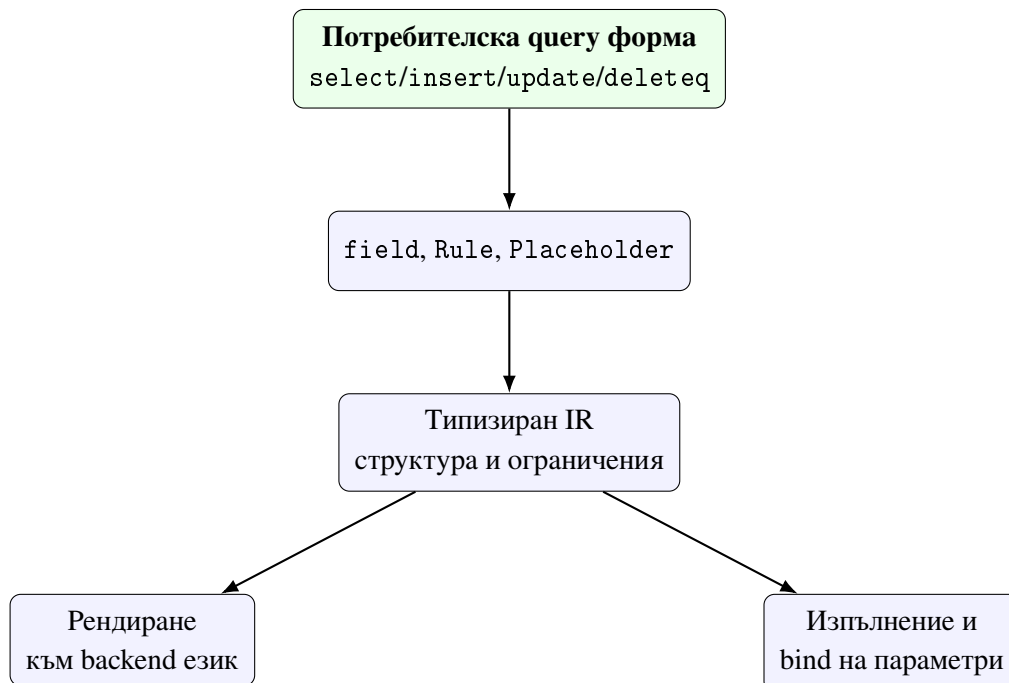
```

3 struct select_query : select_query_tag
4 {
5     using response_type = Response;
6     using row_type = projected_type<Response>;
7
8     template <typename RuleType>
9         requires is_rule<RuleType>
10    [[nodiscard]] constexpr auto where(RuleType r) const
11    {
12        using NewWheres = detail::append_type_t<Wheres, RuleType>;
13        return select_query<Response, Joins, NewWheres, Limits, Groups, Orders>(
14            response_, joins_,
15            detail::tuple_cat(wheres_, detail::orm_tuple<RuleType>(r)),
16            limits_, groups_, orders_);
17    }
18 };

```

Listing 2: Извадка от `select_query` и композицията на `where` клаузи

Listing 2 илюстрира важно теоретично свойство: `query builder`-ът не мутира текст, а изгражда нови типизирани обекти. Това означава, че последователността от операции `select(...).where(...).group_by(...)` може да бъде разглеждана като конструиране на нов тип, а не като поредица от странични ефекти върху низ.

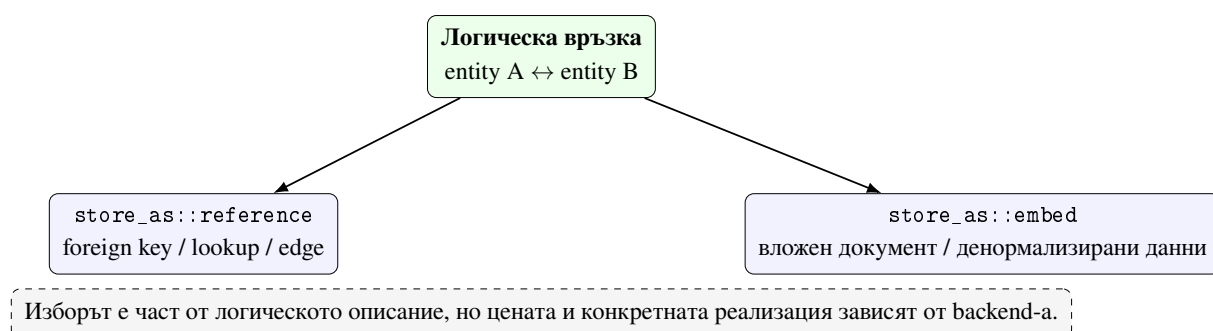


Фигура 4. Pipeline на типизирания `query IR` от приложния `API` до `backend` изпълнението

3.5. Теория на връзките и стратегии за съхранение

Връзките между обекти са един от най-трудните аспекти при обобщаване на ORM извън релационния свят. В релационен контекст една връзка често се материализира чрез външен ключ и JOIN. В документен контекст обаче същата логическа връзка може да се реализира като вграден документ, а в key-value контекст — чрез вторичен lookup или денормализиране.

Библиотеката абстрахира тези варианти чрез `relationship<Strategy, T, Name>` и `store_as::reference/store_as::embed`. `reference` обозначава логическа външна препратка, която backend-ът може да реализира чрез foreign key, lookup, отделен ключ или графова връзка. `embed` обозначава включване на свързаните данни в същата физическа структура. По този начин връзката се описва веднъж логически, а интерпретацията остава задача на конектора.



Фигура 5. Двете основни стратегии за материализация на връзки

Таблица 4. Сравнение между `reference` и `embed` стратегии

Стратегия	Логическа идея	Естествени backend-и	Основен компромис
reference	отделна	SQL, графови	необходим е допълнителен
	идентичност на	системи,	механизъм за навигация или
	свързания обект	key-based lookup	свързване
embed	физическо	документни	по-трудно независимо
	включване в	системи,	обновяване и споделяне на
	родителската структура	денормализирани записи	данни

3.6. Релационен модел

Релационният модел организира данните в таблици, редове и колони. Основните му предимства са ясна схема, декларативен език за заявки и добре дефинирани ограничения върху целостта. За ORM системите това е естествената среда: свойствата на C++ обекта лесно се съпоставят с колони, а връзките — с външни ключове. Операции като JOIN, GROUP BY и транзакции са част от нормалния езиков модел.

Именно релационният backend е естественият базов сценарий, спрямо който може да се оцени до каква степен една ORM библиотека запазва удобството си и извън SQL света. В настоящата разработка този базов сценарий по-късно се реализира основно чрез SQLiteDB, а преносимостта на IR-a се анализира допълнително чрез PostgreSQLDB и MySQLDB.

3.7. Документен модел

Документният модел организира данните в колекции от документи, чиито полета могат да бъдат вложени и не е задължително всички документи да имат еднаква физическа форма. Това го прави естествено съвместим с embed стратегията и по-гъвкав при денормализирани структури. От друга страна, твърде директното пренасяне на релационни абстракции върху документен backend може да бъде подвеждащо. Не всяка релационна операция има еквивалент със същата цена и семантика.

В контекста на настоящата работа документният модел е важен като тест за това дали query IR-ът описва логика, а не синтаксис на конкретен език. Ако един и същ IR може да се интерпретира като BSON-подобен филтър и projection, без потребителят да пише native document заявки, тогава действително е постигнат storage-agnostic слой на абстракция.

3.8. Key-value модел

Key-value системите организират достъпа около ключ и стойност. Те са особено ефективни, когато моделът на достъп е ясен и концентриран около lookup по идентификатор. Недостатъкът е, че общите predicate-базирани заявки и релационните операции са силно ограничени. Затова една ORM абстракция върху key-value backend не може да обещава същата изразителност като върху SQL система.

Този модел е полезен за разграничаване между *логически обект* и *допустим модел на достъп*. Обектът може да бъде един и същ, но не всяка заявка е смислена върху всяка физическа организация. В хранилището това по-късно се вижда ясно при RedisDB, където lookup логиката е естествена, а join-базираният стил е архитектурно неуместен.

3.9. Графов модел

Графовият модел представя данните като възли, ребра и свойства. Силата му е в изразяването на връзки като първокласни обекти, а не като производни от външни ключове. Това го прави подходящ за сценарии със сложни навигации и traversal операции. В същото време графовият модел налага собствен език на заявки и собствена интуиция за селекция и филтриране.

За библиотеката това е показателен сценарий, защото демонстрира, че `connector_trait` може да пренесе общ query IR към MATCH/RETURN/WHERE структура, без потребителят да пише Cypher директно. Ако това е възможно, значи общият слой действително е разположен над конкретния syntax layer.

3.10. Wide-column модел

Wide-column системите, каквато е Cassandra, се основават на разпределени таблици с partition и clustering ключове. Макар синтактично да напомнят релационни таблици, те имат различна теория на достъпа: правилната заявка е тази, която следва физическия ключов модел на разпределение. Следователно наличието на поле в C++ модела не означава автоматично, че може да бъде използвано свободно във WHERE клаузи.

Точно този тип ограничения оправдава capability и constraint подхода на библиотеката. Не е достатъчно query IR-ът да е типово коректен; той трябва да е и допустим спрямо модела на съхранение. По тази причина wide-column моделът е особено ценен за теоретичната защита на compile-time ограниченията.

Таблица 5. Съпоставка между основните модели за съхранение

Модел	Единица за съхранение	Типични връзки	Основно ограничение
Релационен	таблица/ред	foreign key, join	необходимост от съгласувана схема и силна типова дисциплина
Документен	документ	embed или reference	не всяка релационна операция е естествена или евтина
Key-value	ключ и стойност	lookup по ключ, hash полета	силно ограничени predicate заявки извън ключовия достъп
Графов	възел и ребро	traversal връзки	специфичен query език и графова семантика
Wide-column	partitioned row		заявките трябва да следват partition ключово-ориентираната зависимост

3.11. Тестова опора на теоретичните твърдения

Теоретичните твърдения в тази глава не са откъснати от кода. Entity слойът се валидира пряко в `tests/unit/test_property.cpp` и `tests/unit/test_relationship.cpp`, където се проверяват имената на колоните, разпознаването на entity типове, различаването между `store_as::reference` и `store_as::embed`, както и one-to-many inference логиката. Типизираният query IR е покрит в `tests/unit/test_query.cpp`, където се демонстрират field-based предикати, join форми, update composition и delete semantics. Допълнително `tests/unit/test_cpp26_reflection.cpp` показва, че compile-time описанието на полетата може да бъде надградено и чрез reflection-aware path, когато toolchain-ът го позволява.

Тази връзка между теория и тестова опора е съществена. Тя показва, че представените понятия не са свободни академични метафори, а реални архитектурни единици, които могат да бъдат проверявани в изходния код и в автоматизираните тестове.

3.12. Граници на унифицираната абстракция

След изложеното може да се направи важен извод. Унифицираната ORM абстракция не означава, че всички backend-и стават еднакви. Унифицирането е възможно на равнище логически модел, поле, връзка, query IR и базови операции. Отвъд тази граница започват различията: SQL JOIN няма универсален смисъл в Redis; embedding логиката няма идентична цена в класическа SQL таблица; graph traversal не се свежда до обикновен SELECT; partition-driven ограниченията в Cassandra не могат да бъдат маскирани като обикновени predicate правила.

Поради това архитектурно правилният подход не е да се скрият всички различия, а да се направят явни и формално контролирани. Именно тази роля изпълняват `connector_trait`, `capability tags` и `compile-time` ограниченията, разгледани в следващите глави. Точно там ще стане видимо как теоретичната рамка от настоящата глава се превръща в работеща архитектура на библиотеката.

IV. АРХИТЕКТУРА НА БИБЛИОТЕКАТА

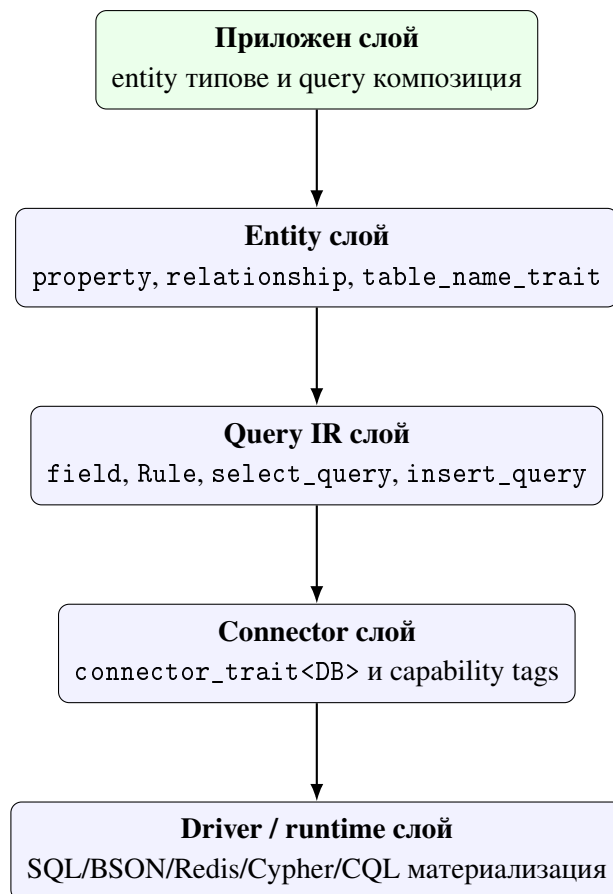
Архитектурата на библиотеката е организирана около ясно разграничение между логическия модел на данните, междинното представяне на заявките и backend-специфичния слой за изпълнение. Това разграничение е ключово за целта на разработката: промяна на базата данни да изисква минимална промяна в приложния код, а в идеалния случай — единствено смяна на типа на конектора. За да бъде постигнато това, архитектурата е изградена от няколко слоя, всеки със собствена отговорност и със строго определени интерфейси.

В теоретичната глава беше показано, че унифицирането е възможно на равнището на логическия модел и query IR-a, но не и чрез игнориране на различията между backend-ите. Настоящата глава прави следващата стъпка: показва как тази теоретична постановка е превърната в конкретна архитектура. Тук библиотеката вече не се разглежда само като идея, а като организирана система от типове, contracts и execution path-ове.

4.1. Архитектурни цели и ограничения

Архитектурата е проектирана при едновременно действие на няколко ограничения. Първо, user-facing API трябва да остане в рамките на идиоматичен C++, без въвеждане на отделен DSL и без runtime reflection механизъм. Второ, query структурата трябва да бъде представима като constexpr обект, така че значима част от валидирането да се извършва от компилатора. Трето, backend слойът трябва да бъде разширяем без наследяване от общ виртуален интерфейс, тъй като подобен интерфейс би скрил различията между backend-ите и би изместил проверките към runtime.

Четвъртото ограничение е свързано със самия multi-backend характер на системата. Библиотеката трябва да може да поддържа SQL и non-SQL конектори без фалшиво уеднаквяване. Това означава, че архитектурата трябва едновременно да допуска общ API и да изразява ограниченията на конкретен backend по формален и проверим начин. Оттук естествено следва изборът на trait-базиран connector слой и capability механизъм.



Фигура 6. Слоеста архитектура на библиотеката

Фигура 6 показва, че библиотеката следва последователна вертикална организация. Приложният код не комуникира директно с драйвера; той работи с C++ типове и query builder-и. Драйверът от своя страна не познава домейн семантиката, а само materialized формата, генерирана от connector слоя. Именно поради това архитектурата може да остане едновременно строга и надграждаема.

4.2. Слоеста организация и разделяне на отговорностите

Най-горният слой е приложният код, който дефинира entity типове, изгражда заявки чрез select, insert, update и deleteq, и ги подава на db<DB>. Следващият слой е логическият модел на библиотеката — entity описанията и query IR. Третият слой е connector abstraction-ът, където connector_trait<DB> материализира същата логика към backend-специфичен синтаксис и поведение. Най-долу стои конкретният драйвер, wire protocol или mock implementation.

Тази архитектура има важен практически ефект. Ако потребителят промени SQLite с MongoDB, query IR остава логически същият, но конекторът го тълкува

като filter/projection изпълнение вместо SQL рендиране. Това не означава, че всеки IR е допустим върху всеки backend; означава, че *формата на абстракцията* е обща, а допустимостта се определя формално чрез capability механизма.

Таблица 6. Основни архитектурни слоеве и техните отговорности

Слой	Основни артефакти	Отговорност
Приложен	entity типове, query builder повиквания	описва домейн логика и формулира операции в идиоматичен C++
Логически	property, relationship, field, Rule	моделира данните и заявките независимо от конкретен backend
Connector	connector_trait<DB>, capability tags	проверява допустимостта и материализира IR-а към backend поведение
Runtime/driver	конкретни connection handle-и и драйверни API-та	изпълнява материализираната заявка и връща резултати

Таблица 6 показва, че библиотеката не използва “голям обект”, който да прави всичко. Вместо това различните решения са локализирани в отделни слоеве. Това е важно както за надграждаемостта, така и за академичния анализ, защото позволява отделните твърдения да бъдат проследени до конкретни типове и модули.

4.3. Entity слой като архитектурна основа

Entity слойът е изграден върху property<T,Name>, relationship<...> и table_name_trait<T>. Той има двойна роля. От една страна, представя C++ домейн модела. От друга страна, служи като логическо описание на схемата или структурата на персистентните данни. Това означава, че архитектурата не прави отделна runtime registry от метаданни. Вместо това типовата система съдържа достатъчно информация, за да бъдат извлечени имена на полета, типове, връзки и целеви контейнер за съхранение.

Особено важен е фактът, че entity слойът е независим от вида база данни. Нито property, нито relationship са SQL-специфични. Те описват логически атрибути и отношения. Така се създава основата за сравнимо поведение на релационни и нерелационни backend-и. На архитектурно ниво именно това прави възможно библиотеката да твърди, че C++ моделът е първичният носител на логическата схема.

4.4. Query IR като централен архитектурен компонент

Query IR е централният архитектурен компонент. Той обединява field референции, предикати, placeholder-и и допълнителни клаузи като `order_by`, `group_by` и `limit`. Вместо да се работи с текст на заявка, библиотеката изгражда типизиран обект, чийто шаблонни параметри кодират логическата структура на операцията.

От архитектурна гледна точка това решение има няколко последствия. Първо, валидирането на структурата се извършва рано. Второ, query IR може да бъде анализиран от connector слоя без загуба на семантична информация. Трето, подготвените заявки могат да запазят не SQL низ, а самото междинно представяне, което е по-естествено за библиотека, ориентирана към C++ типовата система.

Четвърто, query IR-ът е единствената зона, в която се срещат entity описанието и по-късната backend материализация. В този смисъл той е архитектурният “шарнир” на цялата библиотека: достатъчно висок, за да остане storage-agnostic, и достатъчно конкретен, за да може да бъде изпълнен.

4.5. Конекторен слой и формалният contract

Формално connector слойът е реализиран чрез специализация на `connector_trait` `<DB>`. Този подход е аналогичен на стандартни trait шаблони като `std::iterator_traits`. Типът DB може да бъде прост wrapper над connection handle, а цялата ORM логика за него да остане в trait специализацията. По този начин библиотеката не изисква потребителят или авторът на конектора да наследява общ базов клас.

`connector_trait<DB>` определя поне три ключови аспекта: `wire_type`, `cursor_type` и `execute(...)`. По желание специализацията декларира и capability tags като `supports_joins`, `supports_transactions`, `supports_aggregation` или `supports_concurrent_execute`. Наличието или отсъствието на тези псевдо-маркери е достатъчно, за да може по-горният слой да наложи compile-time ограничения.

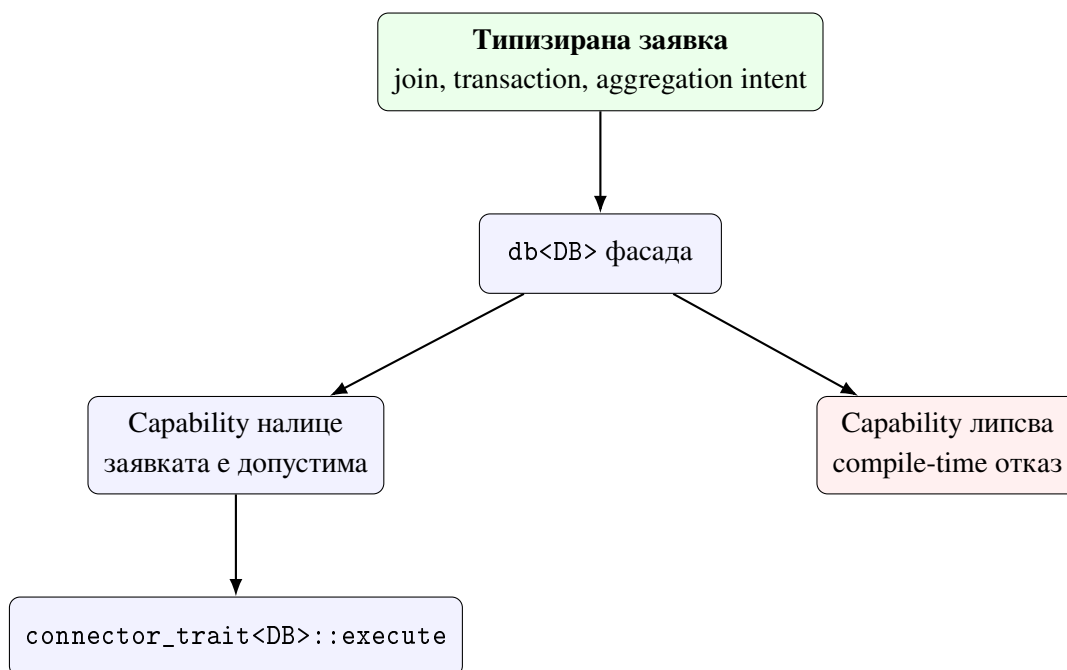
```
1 template <typename DB>
2 concept is_connector = requires {
3     typename connector_trait<DB>::template wire_type<int>::type;
4     typename connector_trait<DB>::cursor_type;
5 };
6
7 template <typename DB, typename Cap>
8 inline constexpr bool has_capability =
```



```
9 | detail::capability_check<DB, Cap>::value;
```

Listing 3: Извадка от формалния connector contract

Listing 3 показва, че contract-ът е структурен, а не наследствен. Това е решаващ архитектурен избор. Ако съществуваше виртуален интерфейс с runtime dispatch, част от типово проверимата информация щеше да бъде изгубена. Вместо това библиотеката предпочита compile-time структура, при която изискванията са видими и за компилатора, и за автора на конектора.



Фигура 7. Capability gating между query IR-a и connector слоя

Фигура 7 е важна, защото показва къде архитектурата взема решение за допустимост. Това не е драйверът и не е runtime логът. Решението се взема в публичния API слой, който вече има достъп до query IR-a и до trait информацията за конектора.

Таблица 7. Роля на основните capability tags в архитектурата

Capability	Архитектурен смисъл
supports_joins	разрешава join-базираните query форми за съответния backend
supports_transactions	позволява използване на transaction_guard и begin/commit/rollback hooks
supports_aggregation	обозначава поддръжка на aggregation операции в логическия query слой
supports_embedding	прави експлицитна native поддръжка на вградени структури
supports_upsert	маркира backend-и, при които upsert е част от естествения execution contract
supports_bulk_insert	обозначава наличието на по-ефективен път за масово записване

4.6. Потребителският фасаден слой db<DB>

Класът db<DB> е публичният фасаден слой. Той капсулира препратка към конкретния backend обект и предоставя уеднаквен API за изпълнение. На практика този слой има две роли. Първата е удобство за потребителя: библиотеката се използва чрез един и същ интерфейс независимо от backend-а. Втората е архитектурен филтър: точно тук се извършват compile-time проверки за capability и съвместимост.

Следователно db<DB> не е просто тънък wrapper над execute. Той е мястото, където библиотеката формализира границата между разрешено и неразрешено поведение за избрания backend. Точно този слой позволява съобщения за грешка от вида “конекторът не поддържа JOIN операции”, вместо грешката да се появи по-късно като неясно runtime поведение.

```

1  template <typename DB>
2      requires is_connector<DB>
3  class db
4  {
5  public:
6      template <typename Query>
7      auto operator<<(Query q)
8      {
9          if constexpr (is_select_query<Query>)
10             {
11                 if constexpr (decltype(q.join_clauses())::size > 0)

```

```

12         {
13             static_assert(has_capability<DB, cap::supports_joins>,
14                 "orm::db: this connector does not support JOIN operations.");
15         }
16     }
17     return connector_trait<DB>::execute(*conn_, std::move(q));
18 }
19 };

```

Listing 4: Извадка от фасадния слой `db<DB>`

Listing 4 показва защо `db<DB>` е повече от удобна фасада. Той е формалното място, където query структурата и connector capability информацията се срещат. Именно тук compile-time архитектурата се превръща в реално ограничение върху API употребата.

4.7. Миграции като архитектурно разширение

Архитектурата включва и прототипен migration слой чрез `migrate<DB>`, `live_schema`, `entity_meta` и DDL операции като `create_table_op`, `add_column_op`, `drop_column_op` и `alter_column_type_op`. Този слой е особено важен от гледна точка на тезата, защото прави явна връзката между C++ модела и реалната схема на съхранение.

Миграционният слой не е универсално завършен за всеки backend, но архитектурно показва, че библиотеката не се ограничава само до query execution. При достатъчно богата специализация на `connector_trait<DB>` entity описанието може да участва и в еволюцията на схемата. Това променя и самия смисъл на ORM слоя: той вече не е само интерфейс към данните, а потенциален източник на schema-aware tooling.

4.8. Архитектурни инварианти

На базата на разгледаните слоеве могат да се формулират няколко инварианта. Първо, приложният код никога не бива да зависи пряко от драйверен API. Второ, query логиката се формулира в типизиран IR, а не в backend-native низове. Трето, разширяването към нов backend се извършва чрез trait специализация и capability декларации, а не чрез промяна на самия query API. Четвърто, ако една операция е недопустима за даден backend, това трябва да стане видимо възможно най-рано — за предпочитане при компилация.

Тези инварианти правят архитектурата едновременно инженерно подредена и академично защитима. Те позволяват всяко следващо разширение да бъде оценявано не само по това дали “работи”, а и по това дали запазва вече формулираните слоеве и

граници.

4.9. Архитектурен извод

Обобщено, архитектурата на библиотеката разделя проблема на три равнища: логически модел, типизиран query IR и backend materialization. Това разделение позволява едновременно строга типова дисциплина, формализирана разширяемост и контролирано поведение върху различни модели за съхранение. В следващата глава това архитектурно решение се разглежда по-конкретно през реализацията на отделните подсистеми, където става ясно как архитектурният план се превръща в изпълним C++ код.

V. РЕАЛИЗАЦИЯ НА ОСНОВНИТЕ ПОДСИСТЕМИ

След като архитектурният модел е дефиниран, следващият въпрос е как той се реализира на ниво конкретни C++ конструкции и как отделните подсистеми си взаимодействат. Настоящата глава разглежда основните building blocks на библиотеката и ги проследява от entity описанията до изпълнението на заявките, подготвените заявки, миграциите и проверката на ограниченията.

Тук фокусът вече не е върху слоевете като архитектурна схема, а върху тяхната реална механика. Как точно се натрупват where клаузите? Как query обектът се запазва за многократна употреба? Как migrate<DB> превръща разлика между entity модел и live_schema в DDL операции? Как нишковият слой гарантира коректна употреба на връзките? Отговорът на тези въпроси показва доколко библиотеката е реално завършена като инженерна система.

5.1. Repository-backed mini case study: User и Post

За да остане изложението обвързано с реалния код, тази глава използва като опорна mini case study типовете User и Post от tests/integration/test_mockdb.cpp. Те са особено полезни, защото съчетават скаларни полета, table naming и join сценарий. Същият пример по-късно се използва и за prepared query, result typing и SQL rendering assertions.

```
1 struct Post
2 {
3     orm::property<int, "id"> id;
4     orm::property<int, "author_id"> author_id;
5     orm::property<std::u8string, "body"> body;
6 };
7
8 struct User
9 {
10    orm::property<int, "id"> id;
11    orm::property<std::u8string, "name"> name;
12    orm::property<double, "score"> score;
13 };
```

Listing 5: Entity типове от tests/integration/test_mockdb.cpp

Listing 5 показва не само как се описва entity моделът, но и как той непосредствено участва в тестовете. table_name_trait<User> и table_name_trait<Post> фиксират логическите контейнери "users" и "posts", а member pointers като &User::id и &Pos

`t::author_id` по-късно участват в query builder-а. Това е добра илюстрация на един от централните мотиви на тезата: моделът, query формата и execution тестовете използват едни и същи compile-time артефакти.

5.2. Скалярни свойства и именуване на контейнерите

Реализацията на `property<T,Name>` свързва стойностен тип с compile-time име. Функцията `column_name()` дава еднозначен достъп до символното име на полето и се използва от конекторите при рендиране към SQL колони, document fields или други backend-специфични понятия. Тази конструкция е едновременно минималистична и достатъчна, защото не въвежда външна регистрация на метаданни.

Именуването на целевия контейнер за съхранение се реализира чрез `table_name_trait<T>`. Макар името да е наследено от ORM традицията, архитектурно то играе по-обща роля: в SQLite и PostgreSQL означава таблица, в MongoDB — collection, в Redis — key prefix, а в Neo4j — label. По този начин библиотеката използва единен логически интерфейс за адресиране на физически различни структури.

Практически това означава, че connector авторът не извлича информация от външна схема или от runtime registry, а от типовата система. Това намалява количеството boilerplate и прави грешките в именуването по-лесни за локализиране в компилационната фаза или в unit тестовете.

5.3. Field wrappers, правила и placeholder-и

Полето в query израз се представя чрез `field<&T::m>`. Това е constexpr wrapper над member pointer, който носи достатъчно информация за типа на таблицата, типа на стойността и името на колоната. Операторите върху `field` изграждат типизирани Rule обекти. Така изрази от вида `field<&User::id>==Placeholder<int>\{\}` не са низове, а compile-time описания на предикати.

Placeholder механизмът има две форми. `Placeholder<T>` моделира анонимни параметри, които се консумират позиционно от `execute()` аргументите. `orm::ph<T,std::placeholders::_N>` моделира индексирани параметри, които позволяват една runtime стойност да се използва на повече от една позиция. Това е особено важно при backend-и като SQLite и PostgreSQL, където има естествена поддръжка на индексно адресирани placeholder-и.

Тестовите `IndexedPlaceholderRendersQuestionMarkN`, `IndexedPlaceholderRenderUsedSameArgInSQL` и `TwoDistinctIndexedPlaceholderRenderCorrectSlots` в `tests/integration/test_mockdb.cpp` показват, че тази подсистема не е теоретична добавка, а реално използван механизъм за изразяване на по-точни `parameter binding` сценарии.

5.4. Изграждане на query обекти

Фабричните функции `select`, `insert`, `update` и `deleteq` създават `query` обекти, върху които последователно се наслагват `where`, `join`, `order_by`, `group_by` и `limit`. Съществено е, че всяка от тези операции не променя низ, а връща нов типизиран обект с разширено вътрешно състояние. В резултат `query builder`-ът е едновременно `fluent` на вид и статичен по природа.

Това позволява в тестовите да се проверява не само крайният резултат от рендирането, а и самата форма на междинното представяне — например броят на `where` клаузите или типът на `response tuple`-а. В `tests/integration/test_mockdb.cpp` това се вижда в `SelectWithInnerJoin`, `SelectWithOrderByDesc`, `SelectWithLimit` и множество други сценарии, които валидират различни фази на `query composition`.

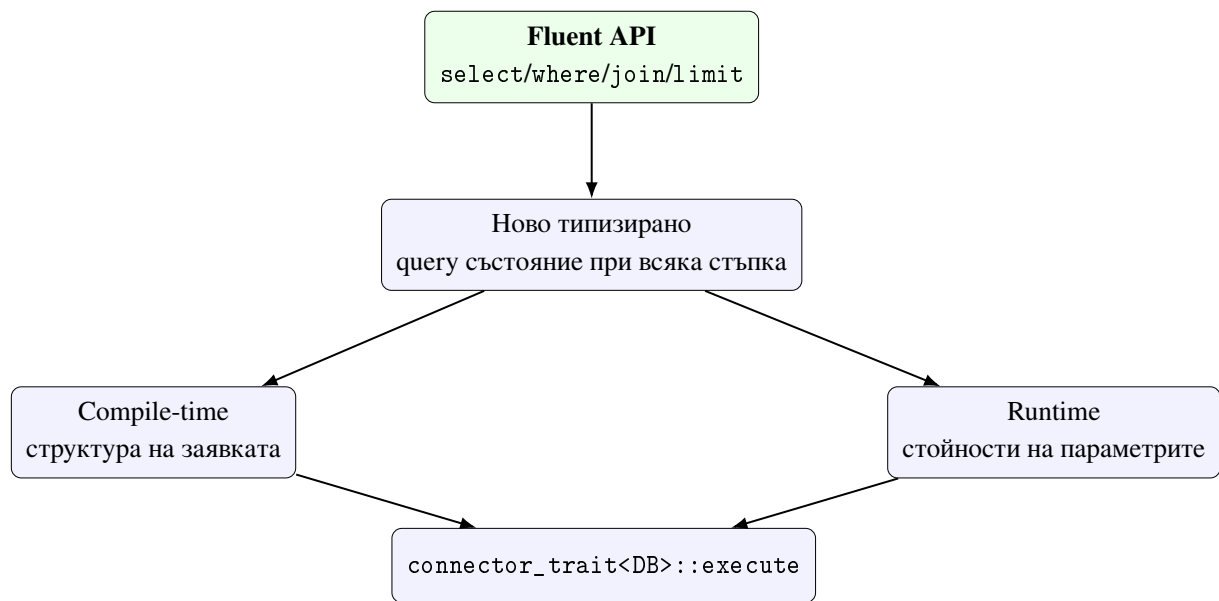
Ключен имплементационен детайл е, че всички `builder` извиквания са *константни изрази*. Вътрешният помощник `orm_tuple::tuple_cat` използва разгъване на пакет чрез индексни последователности вместо `std::tuple_cat`, което прави целия `builder chain constexpr`-съвместим на всички поддържани компилатори, включително `MSVC`. Практическата последица е, че всеки `query` обект може да бъде деклариран `static constexpr`:

```
1 using namespace orm::literals;
2 static constexpr auto get_active =
3     orm::select(orm::field<&User::id>, orm::field<&User::name>)
4         .where(orm::field<&User::score> > 0.0)
5         .where(orm::field<&User::id> == orm::Placeholder<int>{})
6         .limit(10_per_page & 2_page);
```

Listing 6: Query обект деклариран като `compile-time` константа

Операторът `&` комбинира помощниците `_per_page` и `_page` за `user-defined literals` в произволен ред и произвежда `Pagification` обект, чиято структура е известна по време на компилация. Декларирането на заявка като `static constexpr` я превръща в константа с нулева цена за изпълнение: `IR` на заявката се конструира веднъж и никога не се преизгражда

при многократно изпълнение.



Фигура 8. Взаимодействие между query composition, compile-time структура и runtime параметри

Фигура 8 показва, че реализацията работи в два режима едновременно: структурната форма на заявката е фиксирана в типа, докато конкретните стойности се подават отделно. Точно това прави възможно едновременното наличие на compile-time проверки и runtime параметризация.

5.5. Изпълнение, резултати и достъп до полета

На етап изпълнение `db<DB>` предава query IR-a към `connector_trait<DB>::execute`. Резултатът се връща като `orm::result<Row, Fields>`, където `Row` е tuple тип, а `fields` съдържа информация за избраните полета. Това решение позволява както итериране върху материализираните редове, така и достъп чрез `get_field<&T::m>`. Реализацията така остава типобезопасна и след изпълнението на заявката.

Тестовите `SelectResultRowTypeIsTuple`, `SelectMultiFieldRowType`, `SelectGetByMemberPointer` и `SelectGetByIndex` показват, че `result` слойът не е просто контейнер от анонимни редове, а структурирана type-safe абстракция. Това е особено важно за тезата, защото демонстрира, че типовата дисциплина не приключва с query builder-a, а продължава и в result handling-a.

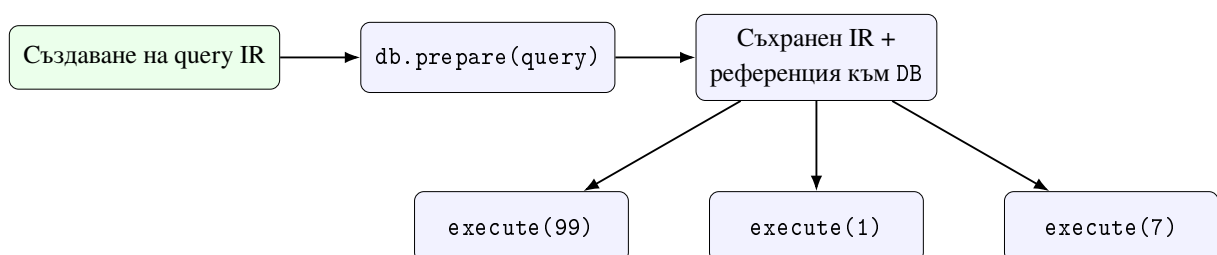
5.6. Prepared queries като повторно използваем execution артефакт

`prepared_query<DB, Query>` разширява изпълнителния модел. Вместо да кешира SQL низ, той съхранява връзка към backend обекта и самия query IR. Това е естествено продължение на compile-time подхода: библиотеката третира query обекта като първичен артефакт, а не като временна стъпка по пътя към низ.

```
1 template <typename DB, typename Query>
2 class prepared_query
3 {
4 public:
5     prepared_query(DB& conn, Query q) : conn_(conn), query_(std::move(q)) {}
6
7     template <typename... Params>
8     auto execute(Params&&... params) const
9     {
10         return connector_trait<DB>::execute(
11             conn_, query_, std::forward<Params>(params)...);
12     }
13
14     [[nodiscard]] const Query& query() const noexcept { return query_; }
15 };
```

Listing 7: Извадка от `prepared_query<DB, Query>`

Listing 7 показва важен инженерен избор: повторната употреба е организирана около IR-a, а не около текстовия резултат от рендирането. Това позволява една и съща логическа заявка да бъде изпълнявана многократно с различни параметри, без да се реконструира builder формата.



Фигура 9. Lifecycle на `prepared_query`: един IR, множество изпълнения

Точно това поведение се валидира от `PrepareReturnsExecutableObject`, `PrepareWithParamExecutesCorrectly`, `PrepareCanBeCalledMultipleTimesWithDifferentParams` и `PrepareExposesQueryIR` в `tests/integration/test_mockdb.cpp`. Тестовите доказват, че `prepared query` обектът е едновременно изпълним и inspectable, което е силно

доказателство за консистентността на подсистемата.

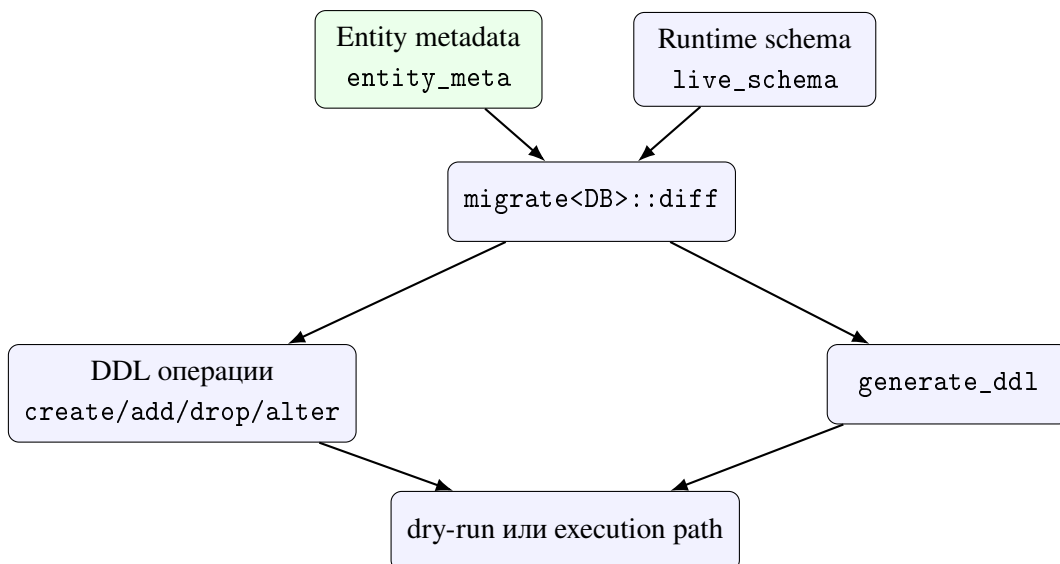
5.7. Миграционен слой и DDL генериране

Реализацията на `migrate<DB>` сравнява описаната от `entity` типовете схема с `live_schema`. Когато има разлика, се създават DDL операции и се подават към `connector_trait<DB>::ddl_for(...)`. Това е добър пример за *clean separation of responsibilities*: `diff` логиката е обща, а конкретният DDL синтаксис е *backend-специфичен*.

```
1 [[nodiscard]] std::vector<ddl_op> diff(  
2     const live_schema& live,  
3     std::span<const entity_meta> entities) const  
4 {  
5     std::vector<ddl_op> ops;  
6  
7     for (const auto& entity : entities)  
8     {  
9         auto live_it = live.find(entity.table_name);  
10        if (live_it == live.end())  
11        {  
12            ops.push_back(create_table_op{entity.table_name, entity.columns});  
13            continue;  
14        }  
15  
16        for (const auto& [col, type] : entity.columns)  
17        {  
18            if (live_it->second.find(col) == live_it->second.end())  
19                ops.push_back(add_column_op{entity.table_name, col, type});  
20        }  
21    }  
22    return ops;  
23 }
```

Listing 8: Извадка от миграционния `diff` pipeline

Интересно е, че именно тук се вижда разликата между *минимален работещ конектор* и *пълен конектор*. Един *backend* може да изпълнява заявки, без да поддържа DDL генериране. Но ако трябва да участва в *schema-aware tooling*, неговият *contract* трябва да бъде разширен.



Фигура 10. Поток на schema migration подсистемата

Тестовите в `tests/unit/test_schema_migration.cpp` покриват създаване на таблица, добавяне и отпадане на колони, dry-run кодове на завършване, генериране на DDL и state machine поведението на `run()`. Това е добър пример как library subsystems с по-високо ниво на отговорност се валидират чрез насочени unit тестове, а не само чрез интеграционен smoke test.

5.8. Нишкова безопасност и transaction helpers

В библиотеката вече съществува отделен слой за `connection_pool<DB,N>`, `thread_local_db<DB>` и `transaction_guard<DB>`. Тези компоненти показват, че реализацията не спира до еднократно изпълнение на заявка, а разглежда и по-реалистични operational сценарии. Важното архитектурно решение е, че достъпът до тези механизми също е capability-базиран. Например `connection_pool` изисква `supports_concurrent_execute`, а `transaction_guard` — `supports_transactions`.

```

1 template <typename DB, std::size_t N>
2 class connection_pool
3 {
4     static_assert(
5         requires { typename connector_trait<DB>::supports_concurrent_execute; },
6         "connection_pool requires connector_trait<DB>::supports_concurrent_execute.");
7
8 public:
9     [[nodiscard]] connection_guard<DB> acquire();
10 };
11

```

```

12 template <typename DB>
13 class transaction_guard
14 {
15     static_assert(
16         requires { typename connector_trait<DB>::supports_transactions; },
17         "transaction_guard requires connector_trait<DB>::supports_transactions.");
18 };

```

Listing 9: Извадка от thread-safety helper слоя

Така compile-time контрактът не се ограничава до query формата, а достига и до lifecycle политиките на конектора. Това е важно за thesis аргумента, че библиотеката не е само builder за заявки, а инфраструктура за систематична и безопасна работа с backend ресурси.

Таблица 8. Основни подсистеми и тяхната тестова опора

Подсистема	Основни файлове	Тестова опора
Entity моделиране	ORM/entity/property.hpp, ORM/entity/relationship.hpp	tests/unit/test_property.cpp, tests/unit/test_relationship.cpp
Query composition	ORM/query/select.hpp и свързаните query headers	tests/unit/test_query.cpp, tests/integration/test_mockdb.cpp
Prepared queries	ORM/connector/prepared_query.hpp	Prepare* сценариите в tests/integration/test_mockdb.cpp
Schema migration	ORM/db/migration/migration.hpp	tests/unit/test_schema_migration.cpp
Thread safety	ThreadSafety/thread_safety.hpp	tests/unit/test_thread_safety.cpp

5.9. Извод за реализацията

Основните подсистеми имат пряка опора в кода и тестовете. tests/unit/test_property.cpp и tests/unit/test_relationship.cpp валидират entity описанията. tests/unit/test_query.cpp доказва, че query builder-ът наистина конструира правилни compile-time структури. tests/integration/test_mockdb.cpp и tests/integration/test_sqlite.cpp валидират изпълнението и prepared query сценариите. tests/unit/t

`est_schema_migration.cpp` покрива diff логиката на миграциите, а `tests/unit/test_thread_safety.cpp` проверява нишковия слой.

Следователно реализацията на основните подсистеми не е само проектно намерение, а архитектурно решение, подкрепено от систематична верификация. Това е и най-важният извод от настоящата глава: compile-time ORM архитектурата в хранилището не остава на равнище абстракция, а се материализира в работещи подсистеми с ясно разграничени отговорности и тестово покритие.

VI. СЦЕНАРИИ ЗА РАБОТА С РЕЛАЦИОННИ И НЕРЕЛАЦИОННИ БАЗИ ДАННИ

Настоящата глава изследва как една и съща C++ абстракция се материализира в различни backend-и. Сценарийният анализ има две цели. Първо, да покаже, че библиотеката не е ограничена до чисто SQL интерпретация. Второ, да изясни границите на унифицираната абстракция и местата, където backend-специфичните ограничения трябва да останат явни.

След теоретичната рамка от Глава III, тук вече не се пита дали различните модели за съхранение са еквивалентни. Вместо това се пита дали една и съща логическа ORM форма може да бъде адаптирана контролирано към тях, без библиотеката да започне да обещава операции, които даден backend не може естествено да изпълнява. Тъкмо в това се проявява стойността на capability-базирания connector contract.

6.1. Методика на анализа

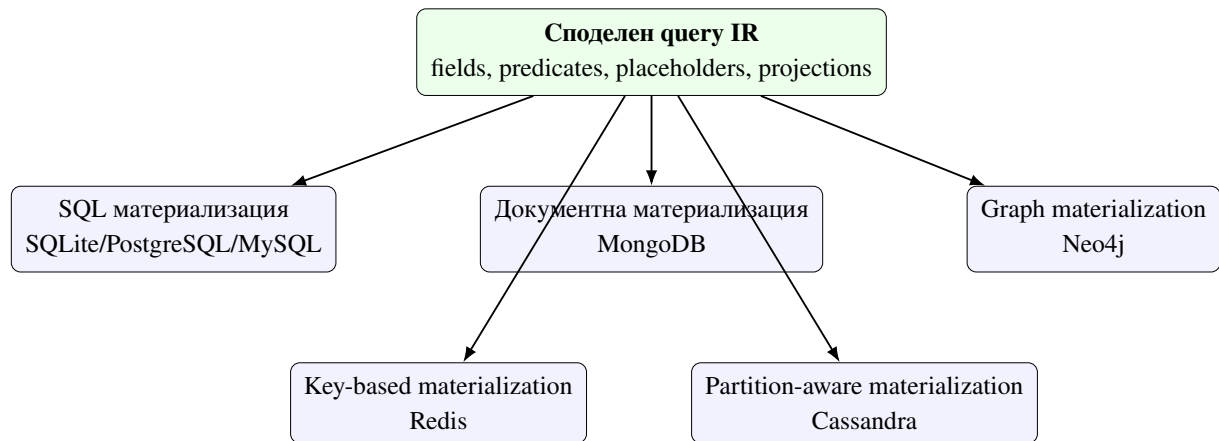
Всеки сценарий следва една и съща последователност: теоретична опора, логическо картографиране от C++ модела, реализация в `connector_trait<DB>`, ограничения и верификация чрез тестове. Това прави сравнението между backend-ите възможно. Не се търси твърдение, че всички системи са еквивалентни, а че една и съща логическа ORM абстракция може да бъде адаптирана контролирано към различни модели за съхранение.

Таблица 9. Еднаква аналитична рамка за всеки backend сценарий

Критерий	Какво се оценява
Теоретична опора	към кой модел за съхранение от Глава III принадлежи backend-ът
Логическо картографиране	как <code>property</code> , <code>field</code> , <code>Rule</code> и <code>table_name_trait</code> се интерпретират в конкретния backend
Sarability профил	кои ORM операции са изрично допустими и кои трябва да бъдат отхвърлени
Execution поведение	как се рендират заявки, параметри и резултатни проекции
Тестова опора	кои unit и integration тестове доказват contract-а в хранилището

Тази аналитична рамка е важна, защото позволява сравнението да остане честно.

SQL backend-ите не се привилегироваха само защото имат по-богат синтаксис, а NoSQL backend-ите не се оценяваха като “непълни” само защото отказваха операции, които са неподходящи за техния модел на съхранение.



Фигура 11. Един споделен query IR и множество backend-specific materialization пътища

Фигура 11 показва ключовото твърдение на главата: не съществуват множество ORM библиотеки в рамките на хранилището, а една логическа библиотека с различни материализационни пътища. Това е силният аргумент в полза на connector architecture-а — backend разнообразието е овладяно чрез общ IR и explicit contract, а не чрез копиране на API-та.

6.2. Релационен базов сценарий: SQLite

Този сценарий стъпва върху релационния модел, разгледан в Глава III. SQLiteD В е най-естественият референтен backend за библиотеката, защото съчетава реална SQL семантика, малък operational контекст и пряка интеграция чрез sqlite3 API. В него property се материализира като колона, table_name_trait — като име на таблица, а Rule дърветата — като WHERE изрази.

Конекторът декларира supports_joins, supports_transactions и supports_aggregation, което го прави най-близък до класическото ORM очакване. Именно върху него се валидират подготвени заявки, bind логика, find_one() сценарии и result hydration към typed tuples. Интеграционните тестове в tests/integration/test_sqlite.cpp показват не само коректност на SELECT, INSERT, UPDATE и DELETE, но и поведение при transaction-capable connector и join-capable profile.

От академична гледна точка SQLite играе ролята на релационен baseline. Ако

дадена ORM форма е проблематична още тук, тя трудно би могла да бъде обобщена към по-екзотични backend-и. Затова настоящата работа използва SQLite като отправна точка, а не просто като един от многото примери.

6.3. Релационна преносимост: PostgreSQL и MySQL

Теоретичната основа и тук е релационният модел, но архитектурният интерес е различен: доколко един и същ query IR остава валиден при различни SQL диалекти и различни правила за bind на параметри. PostgreSQLDB и PostgreSQLLiveDB поддържат `supports_joins`, `supports_transactions` и `supports_aggregation`, а тестовете в `tests/unit/test_postgresql_connector.cpp` и `tests/integration/test_postgresql_live.cpp` показват реално изпълнение срещу жива база данни.

MySQLDB и MySQLLiveDB също декларират `supports_joins` и `supports_transactions`, но тук е важно различието в bind модела. Докато PostgreSQL може естествено да адресира параметри чрез `$1`, `$2`, ..., MySQL разчита на позиционни `?` placeholders. Това не нарушава общия IR, но показва, че logical portability не означава byte-for-byte идентична backend материализация.

Тъкмо тези SQL family сценарии защитават по-силна теза от “работи със SQL”: библиотеката демонстрира, че query IR-ът е достатъчно общ за повече от един диалект, а в същото време е достатъчно честен, за да остави нисконивовите различия на connector слоя.

6.4. Документен сценарий: MongoDB

Този сценарий стъпва върху документния модел от теоретичната глава. При MongoDB property се интерпретира като поле в документ, а query predicate дърветата се превеждат към BSON-подобни филтри. `select` изразите се материализират като `projection`, а таблицата от ORM гледна точка се превръща в `collection`. Точно тук се вижда практическата стойност на storage-agnostic query IR: потребителят не описва `\$eq`, `\$and` и `projection` директно, но конекторът може да ги изведе от същата логическа форма.

Важна подробност е, че `connector_trait<MongoDB>` не декларира SQL-style capability tags. Това не е пропуск, а коректно описание на connector contract-a. Няма претенция, че Mongo backend-ът е join-equivalent на релационните конектори. Вместо това той предлага документно-естествено тълкуване на query IR-a. Unit и integration тестовете

в `tests/unit/test_mongodb_connector.cpp` и `tests/integration/test_mongodb_live.cpp` валидират тази адаптация срещу реален driver слой.

Архитектурната стойност на този сценарий е, че доказва едно важно ограничение: една библиотека може да бъде multi-backend, без да твърди, че всички backend-и поддържат едни и същи операции. Именно този тип “контролирана частичност” прави дизайна научно защитим.

6.5. Key-value сценарий: Redis

Този сценарий стъпва върху key-value теорията. При Redis един и същ entity модел може да бъде материализиран по два основни начина: като SET/GET при еднополеви структури или като HSET/HGETALL при многополеви структури. `table_name()` участва в генерирането на key prefix, а достъпът е концентриран около primary-key equality.

Тук ясно се вижда, че споделеният query IR не означава идентична изразителност. RedisDB умишлено не декларира `supports_joins`, `supports_transactions` или `supports_aggregation`. Това ограничение е част от коректния contract, а не слабост на архитектурата. Unit и live тестовете в `tests/unit/test_redis_connector.cpp` и `tests/integration/test_redis_live.cpp` доказват, че библиотеката правилно материализира key-based достъп, без да обещава неподдържани операции.

Този сценарий е особено ценен за дипломната работа, защото принуждава архитектурата да прави ясно разграничение между *логически модел на данните* и *допустим модел на достъп*. Това разграничение е лесно да се забрави при SQL-first библиотеките, но тук е изведено като централен принцип.

6.6. Графов сценарий: Neo4j

Сценарият за Neo4j стъпва върху графовия модел. Entity типът се материализира като label и набор от свойства, а query IR се превежда към MATCH, RETURN и WHERE фрагменти. Това е важен архитектурен тест, защото graph query семантиката е най-отдалечена от класическия SQL модел. Въпреки това логическите елементи — полета, предикати, филтриране и резултатни проекции — остават разпознаваеми.

Neo4jDB и Neo4jLiveDB декларира `supports_transactions`, но не и join/aggregation capabilities на SQL слоя. Това е смислено: графовият traversal не е същото като SQL join, макар на логическо ниво и двете да свързват данни. Unit и integration

тестовите в `tests/unit/test_neo4j_connector.cpp` и `tests/integration/test_neo4j_live.cpp` показват, че `abstraction`-ът може да адаптира един и същ логически модел и към `graph backend`, ако конекторът поеме отговорността за правилната материализация.

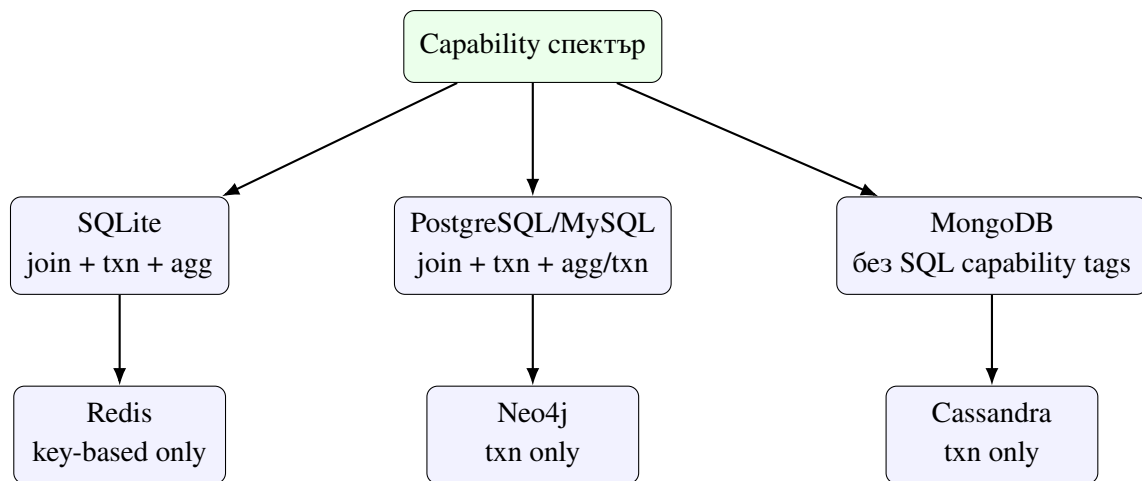
От гледна точка на тезата това е една от най-силните демонстрации, че `query IR`-ът е семантичен, а не синтактичен слой. Ако `IR`-ът беше просто `недо-SQL`, `Neo4j` сценарият би се разпаднал. Фактът, че е възможна работеща графова материализация, подкрепя централната архитектурна претенция.

6.7. Wide-column сценарий: Cassandra

Този сценарий стъпва върху `wide-column` теорията. Макар `Cassandra` да предлага `CQL` синтаксис, наподобяващ `SQL`, действителният модел на достъп е подчинен на `partition` ключовете и на разпределената организация на данните. Затова библиотеката не може да третира `Cassandra` като обикновен `SQL backend`. Наличието на `WHERE` клауза само по себе си не е достатъчно; тя трябва да бъде съвместима с `partition` логиката.

`CassandraDB` и `CassandraLiveDB` декларират `supports_transactions`, но не и `supports_joins`. Това е много показателно. От една страна, `backend`-ът продължава да е част от същата `ORM` библиотека. От друга страна, `contract`-ът категорично отказва операции, които биха били подвеждащи или скъпи в `wide-column` среда. `Unit` и `live` тестовите в `tests/unit/test_cassandra_connector.cpp` и `tests/integration/test_cassandra_live.cpp` дават конкретна верификация на този по-строг `operational profile`.

Именно поради това `Cassandra` е най-добрият пример, че разширяемостта на библиотеката не е базирана на фалшиво уеднаквяване, а на формално описани граници на допустимите операции.



Фигура 12. Сравнителен sarability профил на backend-ите в хранилището

Фигура 12 синтезира едно от най-важните наблюдения в главата: библиотеката не налага еднакъв sarability профил на всички backend-и. Точно обратното — различията се използват като first-class архитектурна информация.

6.8. Съпоставка между test evidence и backend contract

Таблица 10. Backend-и, capability профил и налична тестова опора

Backend	Capability профил	Unit/contract тест	Integration/live тест
SQLite	joins, transactions, aggregation	capability asserts в tests/integr ation/test_sql ite.cpp	tests/integration/test_sqlite.cpp
PostgreSQL	joins, transactions, aggregation	tests/unit/tes t_postgresql_c onnector.cpp	tests/integration/test_postgresql_ live.cpp
MySQL	joins, transactions, aggregation	tests/unit/tes t_mysql_connec tor.cpp	tests/integration/test_mysql_live. cpp
MongoDB	без SQL capability tags	tests/unit/tes t_mongodb_conn ector.cpp	tests/integration/test_mongodb_liv e.cpp
Redis	без joins/ transactions/ aggregation	tests/unit/tes t_redis_connec tor.cpp	tests/integration/test_redis_live. cpp
Neo4j	transactions	tests/unit/tes t_neo4j_connec tor.cpp	tests/integration/test_neo4j_live. cpp
Cassandra	transactions	tests/unit/tes t_cassandra_co nnector.cpp	tests/integration/test_cassandra_l ive.cpp

Таблица 10 е важна, защото превежда сравнението от равнището на narrative analysis към равнището на проверима repository evidence. За всеки backend има поне един ясен contract-level или live-level корелат. Това намалява риска главата да се превърне в декларативен преглед без техническа опора.

6.9. Сравнителен синтез

Съпоставката между backend-ите показва, че библиотеката успява да запази единен логически модел на равнището на entity типове, полета, placeholder-и и query IR. Различията се появяват на равнището на sarability и на физическата организация на данните. Това е и най-важният извод на настоящата глава: успешната multi-backend ORM архитектура не премахва различията между моделите за съхранение, а ги прави контролируеми чрез формален connector contract.

Таблица 11. Съпоставка на backend материализацията

Backend	Основна структура	Логическа интерпретация	Показателно ограничение
SQLite	таблица	пълна SQL ORM интерпретация	стандартна релационна дисциплина и богата query изразителност
	таблица	преносим SQL	различен bind модел и диалектни
PostgreSQL/MySQL		IR	особености
MongoDB	документ	filter/projection	частична еквивалентност спрямо SQL
		интерпретация	операции
Redis	ключ/стойност,	key-based entity	lookup предимно по primary key и липса
	hash	access	на join semantics
Neo4j	възел/свойства	graph-pattern	graph-специфична query семантика вместо
		интерпретация	SQL join слой
Cassandra	wide-column	CQL с ключови	partition-driven достъп и ограничен
	row	ограничения	predicate profile

В резултат multi-backend design-ът на библиотеката може да бъде защитен с по-прецизна формулировка. Системата не обещава “универсална база данни през един API”, а *унифициран логически слой с експлицитни backend граници*. Именно тази формулировка е едновременно инженерно честна и научно обоснована.

VII. ВЕРИФИКАЦИЯ И НАДГРАЖДАЕМОСТ ЧРЕЗ НОВИ КОНЕКТОРИ

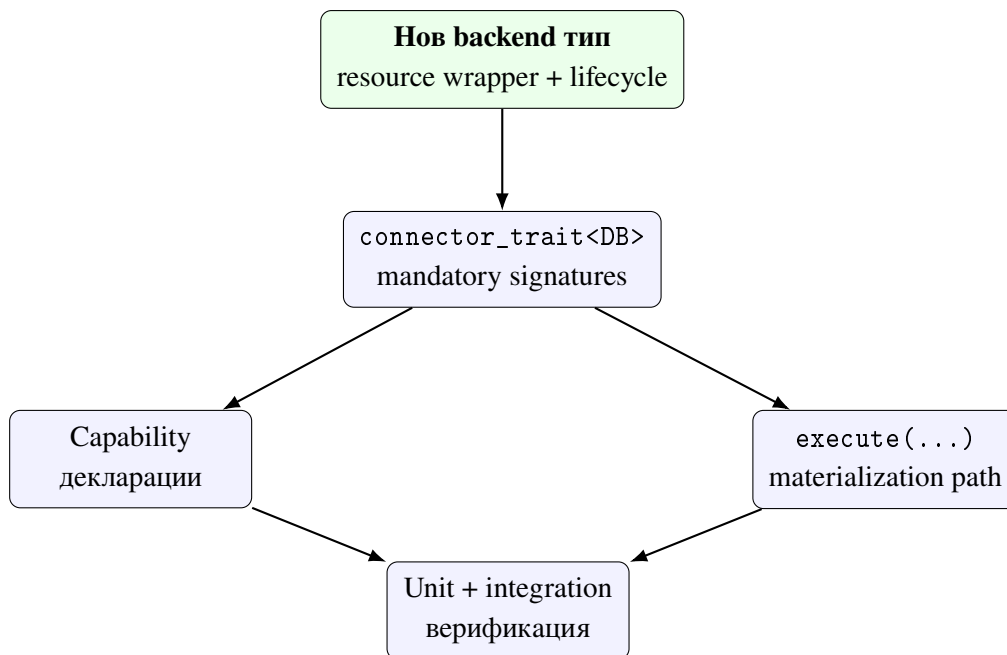
Една от централните претенции на разработката е, че библиотеката е едновременно верифицируема и надграждаема. Това изисква не просто наличие на тестове, а ясно дефиниран `connector contract`, който позволява нов `backend` да бъде добавен без промяна в `query API` и без разпадане на `compile-time` гаранциите.

В предходните глави беше показано, че библиотеката вече работи с релационни и нерелационни `backend`-и. Настоящата глава формализира как това е възможно и как трябва да се добавя следващ `backend`. С други думи, тук надграждаемостта престава да бъде общо качество на дизайна и се превръща в методология с конкретни изисквания, `signatures`, тестови критерии и нива на завършеност.

7.1. Формален `contract` на конекторния слой

Конекторът се описва чрез тип `DB` и специализация `connector_trait<DB>`. Това е структурирен, а не наследствен `contract`. Няма общ виртуален интерфейс `Connector`. Причината е принципна: различните `backend`-и имат различни възможности и ограничения, които трябва да бъдат видими на `compile-time`. Виртуалната йерархия би изравнила твърде много различия и би прехвърлила част от проверките към `runtime`.

Минималният работещ конектор трябва да осигури `wire_type`, `cursor_type` и подходящи `execute(...)` entry points. Допълнителните способности се означават чрез `capability tags`. Така `contract`-ът остава едновременно строг и разширяем. В тази конструкция няма “магически” `registration` слой: компилаторът вижда специализацията директно и може да провери дали тя удовлетворява `is_connector concept-a`.



Фигура 13. Архитектурен workflow за добавяне на нов connector

Фигура 13 показва, че добавянето на backend не е едноетапно действие. То започва с ясно дефиниран resource wrapper, продължава с trait contract, capability профил и execution materialization, и завършва едва след тестова верификация. Това е съществено за дипломната работа, защото поставя надграждаемостта в дисциплиниран инженерен процес.

7.2. Задължителни елементи на минимален конектор

На практика един backend се счита за минимално интегриран, ако:

1. съществува тип, който представлява backend connection wrapper;
2. съществува специализация `connector_trait<DB>` за този тип;
3. специализацията дефинира `template<typenameT>structwire_type;`
4. специализацията дефинира `cursor_type;`
5. специализацията може да изпълнява поне базовите ORM query форми, които твърди, че поддържа.

Това означава, че “конектор” не е просто рендерираща функция, а пълно описание на начина, по който C++ типовете и query IR-ът преминават към конкретния backend. Ако

липсва дори един от тези елементи, добавеният backend не е архитектурно пълноценен, независимо дали част от заявките могат да бъдат изкуствено изпълнени.

Таблица 12. Задължителни и опционални елементи на конектора

Елемент	Роля
DB тип	wrapper над backend handle, session или mock connection
connector_trait<DB>	централна специализация на contract-a
template<typenameT>\new linestruct\wire_type	преобразува C++ тип към wire представяне
cursor_type	описва начина на обхождане или lifecycle-a на резултатите
execute(...) overload-и	материализират query IR-a към backend поведение
Capability tags	ограничават допустимите операции и lifecycle политики
begin/commit/rollback	необходими при transaction support
ddl_for(...)	необходимо при schema-aware интеграция и migrate<DB>
render_constexpr(...)	опционална compile-time materialization hook за специализирани сценарии

7.3. Типизиране, signatures и именуване

Connector contract-ът е строг и по отношение на типовете. `wire_type<T>::typ` е трябва да е стабилна compile-time функция на `T`. `cursor_type` трябва да моделира реалистичен result iteration или поне ясно да обозначава mock поведение. `execute(...)` overload-ите трябва да приемат `DB&` като първи аргумент и да връщат `orm::result<...>` за SELECT форми или `result<std::tuple<>>` за write операции.

Именуването на конекторите в хранилището показва последователна конвенция. “Локални” или unit-friendly конектори използват имена като `SQLiteDB`, `PostgreSQLDB`, `MySQLDB`, `MongoDB`, `RedisDB`, `Neo4jDB` и `CassandraDB`. Реалните live варианти се разграничават експлицитно чрез суфикса `LiveDB`. Това не е дребен стилев избор, а част от contract clarity: още от името е ясно дали даден тип е mockable adapter, in-memory/local wrapper или реален driver-backed backend.

Липсата на наследяване също трябва да се разглежда като формално правило, а не като implementation detail. Нов connector не трябва да наследява “обща ORM

база”, защото това би смесило backend-специфичната логика с runtime polymorphism. Правилният механизъм за разширение е специализацията на `connector_trait<DB>`.

7.4. Sapability механизъм и compile-time ограничения

Sapability tags са основният начин библиотеката да различава backend-ите по допустими операции. Ако `connector_trait<DB>` не декларира `supports_joins`, тогава опит за join-базирана заявка е архитектурно неправилен и следва да бъде отхвърлен още при компилация. Същото важи за транзакции, aggregation и concurrent execution.

От гледна точка на верификацията това е значимо, защото correctness вече не зависи само от тестовите. Част от contract-a е проверим директно от компилатора. Тестовите тогава не доказват съществуването на contract-a, а че конкретните специализации го прилагат последователно. Именно по тази логика `tests/unit/test_mongodb_connector.cpp` и `tests/unit/test_redis_connector.cpp` валидират отсъствието на `supports_joins`, `supports_transactions` и `supports_aggregation` при съответните connector-и.

Тази негативна верификация е също толкова важна, колкото и позитивната. Да докажеш, че backend *не* поддържа дадена операция, е толкова съществено, колкото да докажеш, че поддържа друга.

7.5. Минимален connector skeleton и operational connector skeleton

В хранилището MockDB и MockMigrateDB са особено ценни, защото играят ролята на connector archetypes. Първият показва operational contract-a за query execution, а вторият — как същият contract може да бъде разширен към schema-aware migration сценарии.

```
1  template <>
2  struct connector_trait<MockDB>
3  {
4      using supports_joins          = void;
5      using supports_transactions    = void;
6      using supports_aggregation     = void;
7      using supports_upsert          = void;
8      using supports_bulk_insert     = void;
9      using supports_concurrent_execute = void;
10
11     template <typename T>
12     struct wire_type { using type = T; };
13
```

```

14 struct cursor_type
15 {
16     [[nodiscard]] bool has_next() const noexcept { return false; }
17 };
18
19 template <typename Response, typename Joins, typename Wheres,
20           typename Limits, typename Groups, typename Orders>
21 static auto execute(MockDB& db,
22                     select_query<Response, Joins, Wheres, Limits, Groups, Orders> q)
23     -> result<projected_type<Response>, Response>;
24 };

```

Listing 10: Извадка от connector_trait<MockDB>

Listing 10 е показателен, защото събира в кратка форма почти всички задължителни елементи: capability профил, wire_type, cursor_type и query-specific execute(...) signatures. Освен това connector-ът използва отделни overload-и за select_query, insert_query, update_query и delete_query, което прави mapping-а към backend поведението експлицитен.

```

1 template <>
2 struct connector_trait<MockMigrateDB>
3 {
4     using supports_joins          = void;
5     using supports_transactions    = void;
6     using supports_aggregation    = void;
7     using supports_concurrent_execute = void;
8
9     template <typename T>
10    struct wire_type { using type = T; };
11
12    template <typename... Args>
13    static auto execute(MockMigrateDB& db, Args&&... args)
14    {
15        return connector_trait<MockDB>::execute(
16            static_cast<MockDB&>(db), std::forward<Args>(args)...);
17    }
18
19    [[nodiscard]] static std::string ddl_for(const create_table_op& op);
20    [[nodiscard]] static std::string ddl_for(const add_column_op& op);
21 };

```

Listing 11: Извадка от schema-aware разширението MockMigrateDB

Listing 11 показва как operational connector skeleton-ът може да бъде надграден. Тук query execution логиката се делегира към MockDB, но се добавят DDL hooks и transaction hooks. Това е добър пример за progressive connector maturity модел.

7.6. Транзакции, нишкова безопасност и prepared queries

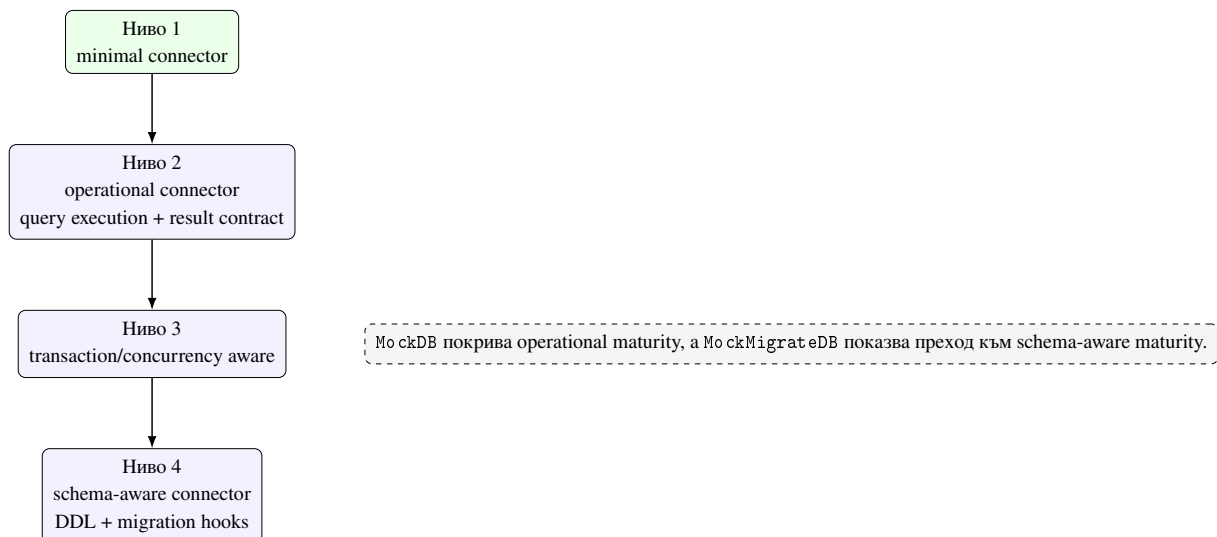
Пълноценният backend contract не се изчерпва с единично изпълнение на SELECT. Ако конекторът декларира `supports_transactions`, от него се очаква да поддържа `begin`, `commit` и `rollback`, така че `transaction_guard<DB>` да бъде валиден. Ако декларира `supports_concurrent_execute`, той трябва да може да участва безопасно в `connection_pool<DB, N>`.

Prepared query слойът също поставя изискване за стабилност на DB& lifetime и за консистентно bind поведение при многократно изпълнение. Следователно зрелият конектор трябва да бъде оценяван не само по това дали рендира правилен синтаксис, а и по operational contract-а около ресурси, транзакции и повторна употреба. Това е особено важно за backend-и, които имат по-сложен драйверен lifecycle от MockDB.

7.7. Миграции и schema-aware интеграция

Конектор, който участва в `migrate<DB>`, трябва да предложи и DDL extension points. В хранилището това е демонстрирано чрез `MockMigrateDB`. Този пример е особено полезен, защото показва как върху вече съществуващ execution backend може да се надгради schema-aware поведение. По този начин библиотеката формализира различни нива на завършеност на конектора.

Тук е важно да се отбележи, че schema-aware поддръжката не е задължителна за всеки backend. Някои конектори могат да бъдат напълно адекватни за query execution анализ, без да поддържат автоматизирани миграции. Но ако конекторът претендира за по-пълна ORM интеграция, наличието на `ddl_for(...)` hooks и migration тестове вече става задължително.



Фигура 14. Стълба на зрелостта за нов connector

Фигура 19 предлага практичен модел за оценка. Тя е полезна и за самото бъдещо развитие на библиотеката, защото позволява новите backend-и да бъдат позиционирани не бинарно като “има/няма connector”, а според нивото на тяхната интеграция.

7.8. Методика за добавяне на нов backend

Добавянето на нов backend следва ясна последователност. Първо се дефинира backend типът и неговият resource wrapper. След това се реализира connector_trait<DB> със задължителните елементи. Следват capability декларации и правилата за рендиране или изпълнение на query IR. Накрая се добавят unit тестове за contract-а и, при възможност, live integration тестове за реалния драйвер.

Архитектурно правилният ред е важен: не се започва от runtime workaround-и, а от формализиране на compile-time границите и на mapping логиката. Ако този ред бъде нарушен, резултатът обикновено е connector, който “работи” за единичен happy path, но не е съвместим с общата архитектура.

Таблица 13. Препоръчителна verification matrix за нов connector

Пласт на верификация	Какво трябва да се докаже	Примерен корелат в хранилището
Concept compliance	is_connector<DB> е удовлетворен	tests/unit/test_mongodb_connector.cpp
Carability honesty	декларираните и недедекларираните carability tags са правилни	tests/unit/test_redis_connector.cpp, tests/integration/test_sqlite.cpp
Execution correctness	query IR-ът се материализира коректно	tests/integration/test_mockdb.cpp и live backend тестове
Resource/lifecycle correctness	transaction и concurrency hooks се държат коректно	tests/unit/test_thread_safety.cpp
Schema-aware correctness	DDL hooks и migration diff-ът работят коректно	tests/unit/test_schema_migration.cpp

7.9. Критерии за „напълно имплементиран и функциониращ“ конектор

На базата на кода и тестове в хранилището могат да се формулират следните критерии:

1. конекторът удовлетворява is_connector contract-a;
2. carability декларации съответстват на реалното поведение;
3. naming и typing конвенциите са съвместими с останалите конектори;
4. ресурсният lifecycle е коректен и проверим;
5. поддържаните ORM операции са покрити от unit тестове;
6. при наличен live driver има integration тестове срещу реална база данни;

7. при заявена schema-aware поддръжка са налични DDL hooks и migration тестове.

Тези критерии са по-строги от обикновено “има работещ адаптер”, защото изискват консистентност между архитектурно намерение, API contract и верифицирано поведение. Именно това различава ad hoc интеграцията от конектор, който наистина принадлежи към библиотеката.

7.10. Верификация чрез тестове

Тестовият слой в хранилището следва структурата на архитектурата. Unit тестовете за отделните конектори валидират `is_connector compliance`, `capability` декларациите и основното рендиране. `tests/unit/test_thread_safety.cpp` проверява `connection_pool`, `thread_local_db` и `transaction_guard`. `tests/unit/test_schema_migration.cpp` валидира migration diff логиката. `tests/integration/test_mockdb.cpp` и live integration тестовете за отделни backend-и доказват, че abstraction-ът работи и при реално изпълнение.

Следователно верификацията не е концентрирана само на едно място. Тя е разпределена така, че всяко архитектурно твърдение да има поне един пряк тестов корелат. Това превръща connector extensibility-то от декларативна цел в проверима инженерна практика.

7.11. Оценка на наличните конектори

SQLiteDB може да се разглежда като референтна зряла реализация за релационен backend. PostgreSQLDB, MySQLDB, MongoDB, RedisDB, Neo4jDB и CassandraDB вече демонстрират важни части от contract-a и имат различна степен на live покритие. MockDB и MockMigrateDB играят особена роля като верификационни конектори: те не са само тестови двойници, а средство за формализиране на правилата на архитектурата.

Оттук следва и по-общият извод на настоящата глава: надграждаемостта на библиотеката не е добавена впоследствие, а е заложена в самия начин, по който е дефиниран connector слой. Именно това позволява разработката да бъде продължавана с нови backend-и, без да се нарушава формалният характер на compile-time ORM архитектурата.

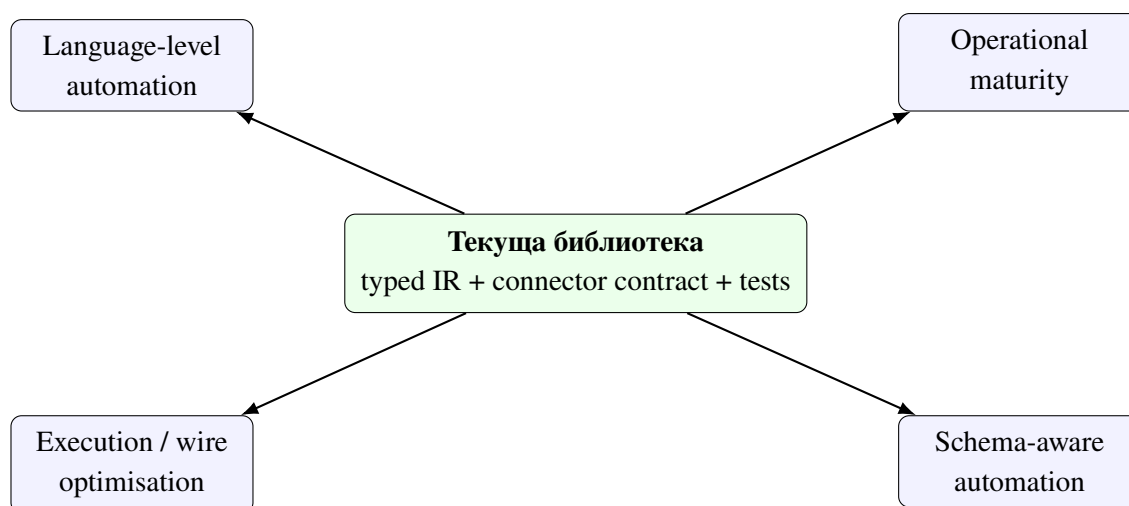
VIII. ОГРАНИЧЕНИЯ И БЪДЕЩО РАЗВИТИЕ

Текущата версия на библиотеката вече демонстрира работеща архитектура за Compile-time Object-Relational Mapping върху релационни и нерелационни backend-и, но научната коректност изисква ясно разграничение между вече постигнатото и следващите стъпки. Библиотеката съдържа реален слой за query IR, sarability-базиран конекторен contract, migration прототип, helper-и за нишкова безопасност и набор от unit и integration тестове. Въпреки това част от подсистемите все още трябва да бъдат разширени или валидирани по-задълбочено в production-like условия.

Бъдещото развитие не е произволен списък от идеи. То следва непосредствено от архитектурните решения, анализирани в предходните глави. Ако C++ моделът е първичен носител на логическата схема, следваща естествена стъпка е по-пълна автоматизация на schema-aware tooling. Ако connector слоят е централна точка на extensibility, тогава бъдещата работа трябва да уточни по-фини sarability класове. Ако query IR-ът е реално споделен слой между различни backend-и, тогава той трябва да бъде изпитан при по-богати operational сценарии и performance ограничения.

8.1. Стратегически направления за развитие

От гледна точка на настоящата дипломна работа бъдещото развитие може да бъде групирено в четири стратегически направления. Първото е *по-голяма operational зрялост* на вече наличните backend-и. Второто е *по-пълна автоматизация на схемата и метаданните*. Третото е *намаляване на runtime overhead-a* чрез по-ниско ниво на изпълнение и по-добър контрол върху wire protocol слоя. Четвъртото е *намаляване на boilerplate-a* чрез по-дълбока интеграция с новите езикови механизми на C++.



Фигура 15. Четири стратегически направления за бъдещо развитие

Фигура 15 е полезна, защото показва, че бъдещата работа не е еднопластова. Част от задачите са насочени към експлоатационна зрялост, други — към по-добра compile-time автоматизация, а трети — към performance и runtime инженеринг. Това разграничение е важно, защото различните направления изискват различни критерии за успех.

8.2. Разширяване на live backend покритието

Макар хранилището вече да съдържа live integration тестове за няколко backend-a, зрелостта на отделните реализации не е еднаква. SQLite остава най-естественият референтен релационен път, защото съчетава най-плътно свързани query rendering, bind логика и result hydration. При PostgreSQL, MySQL, MongoDB, Redis, Neo4j и Cassandra следващата стъпка е по-широко покриване на operational сценарии: по-богати типове, диагностични пътища при грешка, lifecycle политики за връзки и оценка на поведението при по-сложни заявки.

Това направление е важно не само като инженерно подобрене, а и като научна проверка на границите на общия query IR. Само при по-широко емпирично покритие може да се оцени доколко един и същ логически слой остава адекватен за различни типове системи. Например join-базирана логика, която е естествена за PostgreSQL, има съвсем различен operational смисъл в Neo4j или Cassandra. Следователно live покритието не е просто тестова дисциплина, а средство за изследване на архитектурната приложимост.

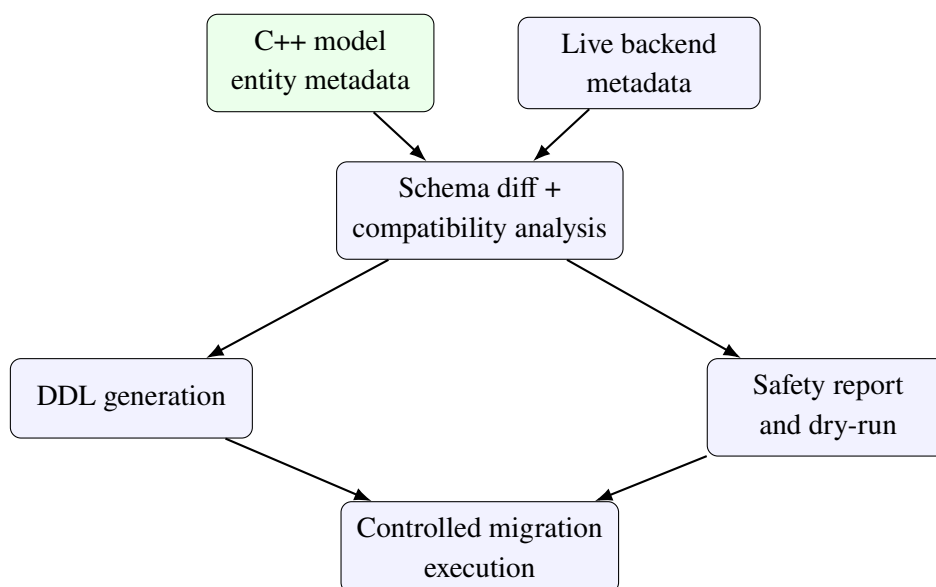
Допълнително, live backend покритието трябва да обхване и негативните пътища: грешки при connection establishment, частично невалидни параметри, driver-specific timeout

сценарии и поведение при прекъсване на връзката. Тези въпроси почти винаги са подценени в ранните ORM библиотеки, но в multi-backend контекст те са решаващи за реалната употреба.

8.3. Пълна schema-aware интеграция и runtime introspection

Прототипният migration слой е достатъчен, за да покаже как C++ entity моделът може да участва в описване на schema evolution. Следващото естествено направление е live introspection на реалната схема, извличане на metadata от драйверите и безопасно изпълнение на DDL операции. Това е особено важно за релационните backend-и, но не е без значение и за документни или wide-column системи, където понятието за schema може да бъде по-гъвкаво или частично.

По-нататъшната работа в тази посока трябва да изясни и границата между compile-time гаранции и runtime факти за вече разположена система. Компиляторът може да валидира query структурата, но не и реалното състояние на production schema без допълнителна runtime информация. Следователно бъдещият migration слой трябва да комбинира два източника на истина: C++ entity описанието и извлечената metadata картина на реалната база.



Фигура 16. Целева посока за schema-aware tooling отвъд текущия migration прототип

Фигура 16 показва, че бъдещият migration pipeline трябва да бъде двупосочен: едновременно да чете от compile-time описанието и от живата система. Това е естествена следваща стъпка, ако библиотеката трябва да бъде използвана не само експериментално,

но и като practical development tool.

8.4. Нишкова безопасност и жизнен цикъл на връзките

Съществуващите helper-и за connection pool, thread-local достъп и transaction guard показват, че библиотеката вече съдържа архитектурна основа за multi-threaded употреба. Бъдещото развитие трябва да отговори на по-конкретни въпроси: как се дефинира `supports_concurrent_execute` за различните драйвери; кога една връзка е безопасна за споделяне; какви са правилата за реконекция, затваряне и отложено освобождаване на ресурси.

Тук има и важен изследователски аспект: различните backend-и имат различна concurrency семантика, а следователно `compile-time capability` обозначаването трябва да бъде достатъчно прецизно, за да не създава подвеждащи обещания. Напълно възможно е бъдещи версии на библиотеката да разделят общото `supports_concurrent_execute` на по-фини `capability` класове като безопасно паралелно четене, паралелно изпълнение с отделни connection handle-и или строга serialization политика.

Практически това означава, че thread-safety helper слой не трябва да остане изолиран помощен модул. Напротив, той трябва да се превърне в архитектурно продължение на connector contract-а и да участва в пълната оценка на зрелостта на даден backend.

8.5. Ниско ниво на изпълнение и wire protocol оптимизации

Експерименталният слой в `WireProtocol/wire_protocol.hpp` показва възможни бъдещи посоки като `io_uring / IOCP awaitable`-и, `zero_copy_result`, `batch_insert` и `make_constexpr_sql`. Тези елементи все още не са production-ready комуникационен стек, но очертават възможност библиотеката да се разшири отвъд класическия synchronous driver модел.

Особено перспективни са нулево-копийната обработка на резултати и `compile-time` генерирането на SQL за конектори, които го позволяват. Те биха приближили библиотеката до още по-нисък runtime overhead, без да се жертва типизираната архитектура. Това е важно, защото една честа критика към богато типизираните abstraction слоеве е, че скриват прекомерен runtime разход. Бъдещата работа трябва да покаже, че `compile-time` изразителността и ниският overhead не са задължително противоположни цели.

Освен това performance темата не трябва да се разглежда само през latency или throughput. Тя включва и memory layout на резултатите, броя временни allocation-и, повторната употреба на prepared query артефакти и качеството на driver-level batching стратегиите.

8.6. Интеграция с C++26 reflection и допълнителна автоматизация

Поддржката на C++26 reflection е вече концептуално подготвена чрез detection логика и helper-и, а `tests/unit/test_cpp26_reflection.cpp` показва наличието на early verification path. Публичният API обаче все още използва експлицитни string literal имена за `property<T, Name>`. Бъдещото развитие трябва да превърне reflection path-a в стабилен публичен механизъм, който позволява автоматично извличане на имената на полетата, без това да нарушава обратната съвместимост.

Подобно развитие би намалило boilerplate-a и би направило още по-видима идеята, че C++ моделът е първичният източник на логическата схема. От изследователска гледна точка то би било интересно и защото би доближило библиотеката до model-first стил, при който голяма част от metadata информацията се извлича автоматично, а не се описва повторно.

Тук обаче има и важна граница. Reflection не трябва да разрушава яснотата на compile-time contract-a. Ако автоматизацията намали прозрачността на типа или затрудни съобщенията за грешка, тя може да отслаби една от основните ценности на библиотеката. Следователно тази посока трябва да се развива внимателно.

8.7. Разширяване на connector contract-a и capability таксономията

С добавянето на нови backend-и ще стане необходимо по-прецизно разграничаване между различни класове възможности. Възможно е например бъдещи версии на библиотеката да разграничават отделно graph traversal capability, partial document embedding support, schema introspection support, streaming result support или explicit batching support. Тази посока следва естествено от текущата trait-базирана архитектура.

По-нататъшната работа трябва да пази едно важно правило: новите capability-та не бива да бъдат декоративни. Те трябва да носят реално compile-time значение и да ограничават недопустимите операции. Ако capability tag няма архитектурно последствие, той не добавя стойност. Точно обратното — прави contract-a по-шумен и по-малко точен.

Таблица 14. Приоритетни направления за бъдещо развитие

Направление	Засегнати подсистеми	Очакван ефект	Основно предизвикателство
Live backend maturity	connector-и, integration tests, error paths	по-висока practical reliability	различна operational семантика между backend-ите
Schema-aware automation	migrate<DB>, DDL hooks, metadata introspection	по-близка до production ORM интеграция	съчетаване на compile-time и runtime истина
Execution optimisation	wire protocol layer, prepared queries, batching	по-нисък runtime overhead	запазване на typed abstraction-a
Reflection-driven modelling	entity layer, metadata extraction, API ergonomics	по-малко boilerplate и по-автоматичен model-first подход	баланс между автоматизация и прозрачност
Capability refinement	connector_trait<DB> и static checks	по-точен contract за нови backend-и	избягване на декоративни capability tag-ове

8.8. Необходимост от по-строга емпирична оценка

Една бъдеща версия на работата следва да включва и по-систематичен benchmarking слой. Той трябва да сравнява не само отделни backend-и, но и различни execution режими вътре в библиотеката — директно изпълнение, `prepare()` + reuse, batch insert сценарии и различни форми на result materialization. Такъв benchmarking би бил ценен както инженерно, така и научно, защото би показал реалната цена на различните архитектурни избори.

Наред с това, полезно би било да се добави и формализирана оценка на error diagnostics. Една от силните страни на compile-time ORM дизайна е именно ранното локализиране на грешки. Следователно качеството на съобщенията за грешка и яснотата на compile-time failure пътищата също са валиден обект на изследване.

8.9. Обобщение

Бъдещото развитие на библиотеката не започва от празно място. Напротив, то стъпва върху вече работеща архитектура, която е достатъчно обща, за да поеме нови backend-и, и достатъчно строга, за да наложи формални ограничения. Именно това прави разработката подходяща както за продължаване като инженерна система, така и за по-нататъшни академични изследвания в областта на compile-time моделирането на достъп до данни.

Най-важното заключение от настоящата глава е, че бъдещата работа не трябва да се разбира като поправка на несъстоятелен прототип. Тя е естествено разгръщане на вече защитима архитектурна основа. Това е съществено различие: библиотеката е достигнала ниво, на което разширението е въпрос на дисциплинирано развитие, а не на концептуално преосмисляне.

IX. ЗАКЛЮЧЕНИЕ

Настоящата дипломна работа представи проектирането, реализацията и анализа на Compile-time Object-Relational Mapping библиотека за C++, ориентирана към работа както с релационни, така и с нерелационни бази данни. В центъра на разработката стои идеята, че query структурата, логическият модел на данните и съществена част от ограниченията върху употребата на backend-ите могат да бъдат описани и валидирани чрез типовата система на езика.

С това работата адресира един конкретен проблем от съвременната практика: разминаването между силно типизиран приложен код и хетерогенните модели за съхранение, които този код трябва да използва. Класическите ORM решения са най-силни в рамките на релационния свят, а чисто driver-oriented подходите често прехвърлят твърде голяма част от коректността към runtime. Настоящата разработка показва, че между тези два полюса съществува жизнеспособна архитектурна позиция — compile-time ORM слой, който е едновременно практичен, формално структуриран и достатъчно честен по отношение на backend различията.

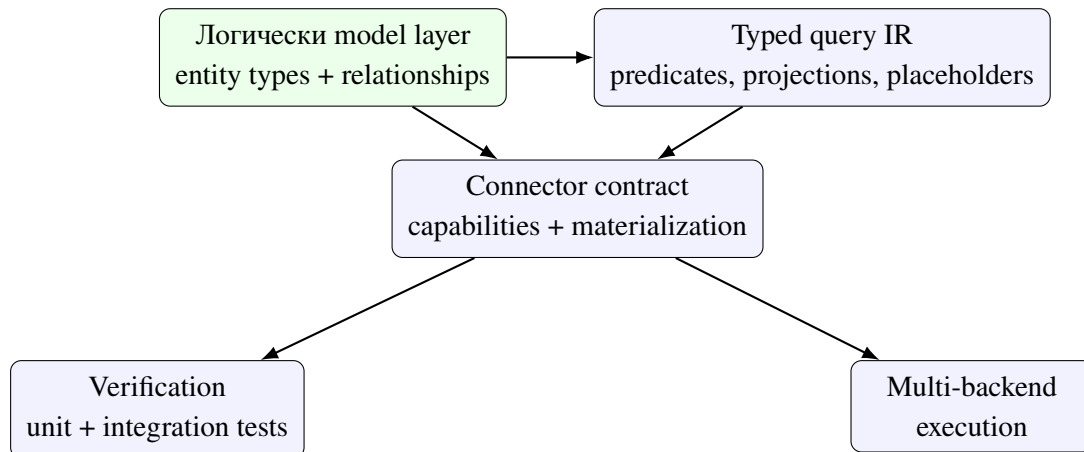
9.1. Обобщение на решената задача

Поставената задача не беше просто да се изгради още един удобен query builder. Целта беше да се проектира библиотека, в която:

1. логическият модел на данните се описва чрез C++ типове;
2. заявките се представят като типизиран IR, а не като низове;
3. backend-ите се интегрират чрез формален connector contract;
4. различията между storage моделите се управляват чрез capability и constraint механизми;
5. коректността се подкрепя едновременно от compile-time проверки и от тестова инфраструктура.

В рамките на дипломната работа всяка от тези подцели беше адресирана с конкретни архитектурни и implementation решения. Entity слойът въведе типизирани

свойства и отношения, политика за съхранение чрез `store_as` и `traits` за именуване на логическите контейнери. Query builder-ът беше организиран около типизиран `intermediate representation`. Connector слойът беше формализиран чрез `connector_trait<DB>`, а verification стратегията бе подкрепена от unit и integration тестове за множество backend-и.



Фигура 17. Синтез на основната конструкция на разработената библиотека

Фигура 17 обобщава цялостната логика на дипломната работа. Тя показва, че библиотеката не е съвкупност от отделни техники, а последователна система, в която model layer, query IR, connector contract, verification и multi-backend execution се поддържат взаимно.

9.2. Обобщение на постигнатите резултати

Разработката показва, че C++ предоставя достатъчно силни механизми — шаблони, `constexpr`, `concepts` и `trait` специализации — за изграждане на ORM слой, в който заявките не се описват като низове. Вместо това те се представят чрез типизиран query IR, който може да бъде анализиран на compile-time и материализиран по различен начин според backend-a.

На равнище модел на данните библиотеката демонстрира как `property`, `relationship`, `store_as` и `table_name_trait` могат да играят ролята на единен логически слой. Този слой не е ограничен до релационната интерпретация, а служи като общо описание на данните и връзките между тях. Именно това позволява една и съща application-level структура да бъде пренасяна към таблици, документи, ключове и стойности, графови възли или wide-column редове.

На равнище архитектура `connector trait`-ът е формализиран като централен

механизъм за разширяемост. Именно чрез него библиотеката адаптира един и същ query IR към SQL, BSON-подобни филтри, Redis команди, Cypher и CQL. Capability механизмът допълва тази архитектура, като прави недопустимите операции видими още при компилация.

На равнище верификация разработката се опира на систематичен набор от unit и integration тестове, които покриват entity слоя, query builder-a, prepared query механизма, миграциите, нишкова безопасност и конкретните конектори. По този начин архитектурните твърдения са подкрепени с реални доказателства от кода и тестовата инфраструктура.

9.3. Научни и приложни приноси

Първият принос на работата е демонстрацията, че practically useful Compile-time Object-Relational Mapping библиотека може да бъде реализирана в стандартен C++. Това се постига без външна кодогенерация, без виртуален connector интерфейс и без принуждаване на потребителя да напуска идиоматичния C++ модел.

Вторият принос е в изясняването на връзката между C++ entity описанието и физическата организация на данните. Показано е, че C++ моделът може да бъде третиран като първичен носител на логическата схема, докато конкретният backend определя начина на материализация. Тази постановка има стойност не само за ORM библиотеките, а и по-общо за системите, които се опитват да комбинират силна статична типизация с хетерогенни източници на данни.

Третият принос е формализирането на connector contract, който обединява разширяемост и строга compile-time дисциплина. Този contract прави възможно надграждането на библиотеката с нови backend-и, без да се разрушават вече съществуващите гаранции в query API-то. Точно тук работата се различава от подходи, които предлагат общ интерфейс, но скриват критичните разлики между backend-ите.

Четвъртият принос е показването, че verification-ът на подобна библиотека трябва да бъде многостепенен. Не е достатъчно само “да има тестове” или само “да има compile-time checks”. Необходима е комбинация от двете: concept-level constraints, capability assertions, query rendering tests, live integration сценарии и migration/thread-safety проверки.

Таблица 15. Връзка между целите на дипломната работа и реализираните резултати

Цел	Реализиран механизъм	Доказателствена опора
Типизиран модел на данните	property, relationship, table_name_trait	entity headers и unit тестовете за property/relationship
Типизиран query слой	query builder-и и typed IR	query headers, tests/unit/test_query.cpp, tests/integration/test_mockdb.cpp
Разширяем multi-backend слой	connector contract и capability tags	connector headers и backend-specific unit/live тестове
Schema-aware посока	migrate<DB> и DDL hooks	tests/unit/test_schema_migration.cpp
Безопасна operational употреба	thread-safety и transaction helper-и	tests/unit/test_thread_safety.cpp

Таблица 15 показва, че дипломната работа не остава на равнището на декларативен design document. За всяка основна цел има конкретен реализиран механизъм и поне един пряк технически корелат в хранилището.

9.4. Основни ограничения и граници на обобщението

Работата също така ясно показва, че multi-backend ORM абстракцията има граници. Не всеки модел за съхранение предлага едни и същи операции, а следователно общият query IR не може да бъде интерпретиран еднакво навсякъде. Именно затова capability и constraint механизмите са толкова важни. Те не отслабват библиотеката, а я правят коректна спрямо различните backend-и.

Допълнително, макар архитектурната основа за миграции, нишкова безопасност и по-ниско ниво на изпълнение вече да съществува, тези подсистеми остават естествено поле за по-нататъшно развитие и емпирично разширяване. Това означава, че работата трябва да бъде оценявана като завършена архитектурна и implementation демонстрация, но не и като последна дума по темата за compile-time достъп до данни.

От академична гледна точка е важно да се подчертае и още една граница.

Настоящата разработка не доказва, че `compile-time ORM` е универсално най-добрият подход за всички приложения. Тя доказва нещо по-точно: че за определен клас системи — системи с нужда от ясна статична структура, висока `type safety` и `controlled multi-backend extensibility` — този подход е жизнеспособен и инженерно защитим.

9.5. Значение за практиката и за бъдещите изследвания

Практическата стойност на разработката се състои в това, че библиотеката вече осигурява стабилна основа за по-нататъшно инженерно развитие. `Entity` описанието, `query IR`-ът, `connector contract`-ът и `verification` стратегията са достатъчно ясно формулирани, за да позволят дисциплинирано добавяне на нови `backend`-и и `operational` възможности.

Изследователската стойност е свързана с по-общия въпрос за ролята на типовата система в моделирането на достъпа до данни. Работата показва, че `type system`-ът на `C++` може да служи не само за описание на данни и алгоритми, но и за формализиране на архитектурни граници, допустимост на операции и преход между логически и физически слоеве. Това поставя библиотеката в по-широк контекст, който надхвърля `ORM` темата в тесния смисъл.

9.6. Заключение бележки

Направеният анализ потвърждава, че типовата система на `C++` е достатъчно мощна, за да служи не само за описване на данни и алгоритми, но и за формализиране на достъпа до персистентни системи. Комбинацията от статичен `entity` модел, `query IR`, `trait`-базиран `connector` слой и систематична верификация показва устойчив път към библиотеки, които са едновременно изразителни, безопасни и надграждаеми.

По този начин дипломната работа постига както инженерна, така и изследователска цел: изгражда работеща библиотека и едновременно формулира принципи, които могат да бъдат използвани при бъдещи разработки на `compile-time` слоеве за достъп до данни. Най-същественият извод е, че `compile-time ORM` подходът не е просто теоретично упражнение, а реална инженерна алтернатива за системи, които изискват строгост, яснота и разширяемост при работа с множество модели за съхранение.

ЛІТЕРАТУРА

- [1] Code Synthesis Tools CC. ODB: C++ object persistence framework. <https://www.codesynthesis.com/products/odb/>, 2010. Online documentation.
- [2] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [3] D. Richard Hipp. SQLite: A self-contained, serverless, zero-configuration, transactional SQL database engine. <https://www.sqlite.org/>, 2000. Online documentation.
- [4] D. Richard Hipp. SQLite C/C++ interface – an introduction. <https://www.sqlite.org/cintro.html>, 2000. Online documentation.
- [5] Maciej Hryniuk and Mateusz Bao. SOCI: The c++ database access library. <https://soci.sourceforge.net/>, 2004. Online documentation.
- [6] ISO/IEC JTC1/SC22/WG21. ISO/IEC 14882:2024 – programming language C++. International standard, International Organization for Standardization, 2024.
- [7] Oracle Corporation. MySQL connector/C – C client library for MySQL. <https://dev.mysql.com/doc/c-api/en/>, 2000. Online documentation.
- [8] The PostgreSQL Global Development Group. libpq — C library for PostgreSQL. <https://www.postgresql.org/docs/current/libpq.html>, 1996. Online documentation.
- [9] Roland Witt. sqlpp11: A type safe embedded domain specific language for SQL queries and results in C++. <https://github.com/rbock/sqlpp11>, 2014. GitHub repository.

Х. РЕЧНИК НА ИЗПОЛЗВАНИТЕ СЪКРАЩЕНИЯ И ПОНЯТИЯ

Настоящият раздел има две функции. От една страна, той обобщава използваните съкращения и термини, за да улесни последователното четене на текста. От друга, той фиксира точния смисъл, в който някои понятия се употребяват в рамките на дипломната работа. Това е важно, защото думи като “модел”, “конектор”, “migration” или “capability” могат да имат различни значения в зависимост от контекста.

10.1. Основни съкращения

Таблица 16 обобщава основните съкращения и термини, използвани в дипломната работа.

Таблица 16. Речник на използваните съкращения

Съкращение	Пълна форма	Контекст
API	Application Programming Interface	Програмен интерфейс между софтуерни компоненти
BSON	Binary JSON	Бинарен формат за сериализация, използван от MongoDB
CQL	Cassandra Query Language	Език за заявки към Apache Cassandra
CRUD	Create, Read, Update, Delete	Четири основни операции върху данни
DDL	Data Definition Language	Език за описание и промяна на схема
IR	Intermediate Representation	Междинно представяне на query структурата
ORM	Object-Relational Mapping	Картографиране между обектен и персистентен модел
PFR	Plain Fields Reflection	Boost . PFR механизъм за работа с агрегати
RAII	Resource Acquisition Is Initialization	Идиом за управление на ресурси в C++
RDBMS	Relational Database Management System	Релационна система за управление на бази данни
SFINAE	Substitution Failure Is Not An Error	Класически механизъм за шаблонни ограничения в C++
SQL	Structured Query Language	Стандартизиран език за релационни заявки
TMP	Template Metaprogramming	Мета-програмиране чрез шаблони
WG21	Working Group 21	Работна група към ISO за стандарта на C++

10.2. Работни дефиниции на ключови понятия

В рамките на дипломната работа няколко термина имат по-специфичен смисъл от общоупотребимия. Таблица 17 фиксира този смисъл, за да не се смесват архитектурните и общите значения на понятията.

Таблица 17. Работни дефиниции на ключовите понятия в дипломната работа

Понятие	Работна дефиниция
Логически модел	compile-time описанието на entity типовете, техните свойства и връзки, независимо от физическия backend
Query IR	типизираното междинно представяне на заявката преди конкретната материализация към SQL, BSON-подобен филтър, Redis команда, Cypher или CQL
Connector contract	формалният набор от изисквания към <code>connector_trait<DB></code> , чрез който backend-ът участва в системата
Capability tag	compile-time маркер, който ограничава допустимите операции и изразява архитектурни възможности на backend-a
Prepared query	обект, който свързва предварително конструиран query IR с конкретна <code>db<DB></code> инстанция за многократно изпълнение
Schema-aware поддръжка	ниво на интеграция, при което библиотеката може да работи със <code>schema metadata</code> , <code>DDL hooks</code> и <code>migration diff</code> логика
Hydration	материализиране на резултатите от backend-a в типизиран C++ резултатен обект
Verification	съвместната роля на compile-time ограниченията, unit тестовете и integration тестовете за доказване на коректност

10.3. Нотация и именуване в текста

Дипломната работа използва и последователна нотация за означаване на типове, traits и backend роли. Тази нотация не е случайна; тя отразява реалната структура на хранилището и улеснява връзката между текста и кода.

Таблица 18. Нотация и именуване, използвани в текста

Означение	Смисъл в дипломната работа
DB	generic backend тип, използван като параметър в traits, helper-и и db<D B> обвивката
db<DB>	фасаден тип, който предоставя user-facing интерфейса за изпълнение на заявки
connector_trait	специализацията, която описва как конкретният backend изпълнява query IR-a
wire_type<T>	mapping от C++ тип към wire-level представяне за конкретния backend
cursor_type	абстракция за lifecycle-a или обхождането на резултатите
field<...>	compile-time обозначение на избрано property поле в query API-to
Placeholder<T>	анонимен runtime параметър в query IR-a
*LiveDB	naming convention за конектори, които използват реален driver-backed backend, а не mock вариант

10.4. Репозиториен контекст на най-често цитираните артефакти

Тъй като дипломната работа е силно обвързана с реалното хранилище, полезно е да се фиксира и ролята на най-често цитираните файлови групи. Таблица 19 служи като кратък ориентир за това къде в проекта се намират основните доказателствени източници.

Таблица 19. Карта на основните репозиторийни артефакти, използвани в текста

Път или група файлове	Роля	Как се използва в дипломната работа
lib/include/ORM/entity/	entity модел	използва се като източник за property, relationship и логическото описание на данните
lib/include/ORM/query/	query IR и fluent API	използва се за анализ на select, insert, update, delete и query composition слоя
lib/include/ORM/connector/	connector contract и execution layer	използва се за формализиране на capability механизма, prepared queries и db<DB> фасадата
lib/include/ORM/db/migration/	migration и schema-aware логика	използва се като база за анализа на DDL hooks и migration diff pipeline-a
tests/unit/	unit verification	служи като доказателствен слой за локалната коректност на отделните подсистеми
tests/integration/	integration verification	служи за демонстрация, че абстракцията работи при реално или mock изпълнение на заявки
doc/v2/bg/ и doc/v2/en/	документационен слой	използват се за синхронно академично представяне на архитектурата и резултатите

10.5. Извод

Разширеният речник показва, че терминологичната последователност е неразделна част от качеството на настоящата дипломна работа. Когато архитектурата е силно типизирана и multi-backend насочена, прецизната употреба на термините не е само стилъв въпрос, а условие за коректно разбиране на цялата аргументация.

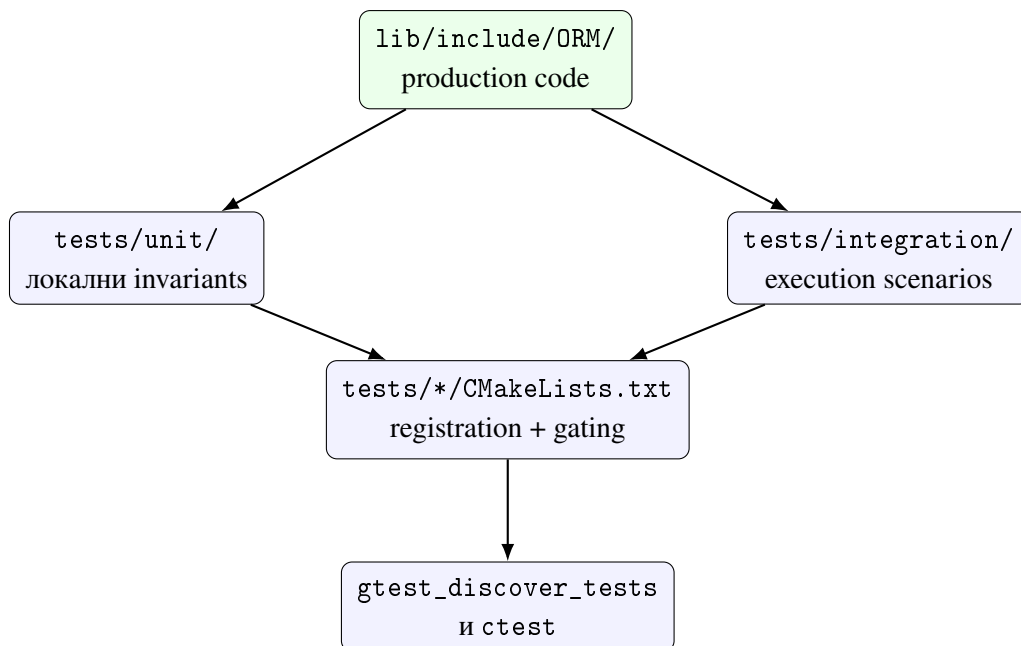
XI. КАРТА НА ВЕРИФИКАЦИОННИТЕ СЦЕНАРИИ И РЕПОЗИТОРИЙНИТЕ АРТЕФАКТИ

Настоящото приложение систематизира връзката между архитектурните твърдения в дипломната работа и конкретните артефакти в хранилището. Целта му не е да дублира вече изложените аргументи, а да предостави компактна и проверима карта на това къде в кода и тестовата инфраструктура се намират основните доказателства за коректност.

Подобно приложение е особено полезно в контекста на настоящата разработка, защото библиотеката не е просто концептуален prototype. Тя е реално изградена като CMake проект със source код, unit тестове, integration тестове и conditional live тестове за различни backend-и. Именно тази материална структура позволява на дипломната работа да премине отвъд ниво “архитектурно намерение” и да бъде защитена като инженерно верифицируема система.

11.1. Обща структура на доказателствения слой

Верификацията в хранилището следва архитектурната структура на библиотеката. Production кодът е концентриран главно под `lib/include/ORM/`, а тестовете са разделени между локален unit слой и `integration/live` слой. CMake файловете регистрират тези тестове по различен начин в зависимост от това дали даден backend е винаги наличен, условно наличен или изисква външна среда.



Фигура 18. Верификационен слой на хранилището и поток към изпълнимите тестове

Фигура 18 показва защо verification-ът не трябва да се възприема като единичен тестов пакет. Той е съставен от няколко нива: локални свойства на типовете, contract-но поведение на connector-ите, интеграционни сценарии за query execution и build-level механизъм, който решава кои тестове са налични в конкретната среда.

11.2. Каталог на основните unit тестове

Unit тестовете покриват две различни категории знания. Първата категория е свързана с логическия модел и с основните generic механизми на библиотеката. Втората категория е свързана с backend-специфичните connector-и и техните capability профили.

Таблица 20. Основни unit тестове за generic и core подсистемите

Файл	Подсистема	Какво валидира
test_types.cpp	базови типови alias-и	коректност на публичните alias-и, включително chrono-related типове
test_property.cpp	entity property layer	compile-time модел на скаларните полета и type trait поведението
test_relationships.cpp	entity relationship layer	infer логика за връзките и storage strategy поведението
test_query.cpp	query IR и fluent API	коректност на select, insert, update, delete и производните query структури
test_thread_safety.cpp	concurrency helpers	поведение на connection pool, thread-local достъп и transaction guard механизма
test_wire_protocol.cpp	experimental execution layer	ниско ниво на compile-time/wire-oriented механизми
test_cpp26_reflection.cpp	future language integration	early path за reflection-базирана автоматизация
test_schema_migration.cpp	migration layer	DDL generation, diff логика и state machine поведение

Таблица 20 е важна, защото показва, че unit слой не се изчерпва с connector тестове. Той покрива и generic архитектурните елементи, върху които по-късно стъпват всички backend-и.

Таблица 21. Backend-oriented unit тестове и техният contract focus

Файл	Backend фокус	Тип доказателство
test_postgresql_connector.cpp	PostgreSQLDB	is_connector compliance, capability asserts и SQL parameter rendering
test_mysql_connector.cpp	MySQLDB	joins/transactions capability профил и MySQL-specific rendering expectations
test_mongodb_connector.cpp	MongoDB	negative capability validation и BSON-like filter rendering
test_redis_connector.cpp	RedisDB	absence of joins/transactions/aggregation и key-value command materialization
test_cassandra_connector.cpp	CassandraDB	capability ограничения и CQL rendering корелати
test_neo4j_connector.cpp	Neo4jDB	transaction capability и graph-oriented query materialization

11.3. Integration и live тестове

Integration слойт е организиран около два принципа. Първият е да има backend-независим mock baseline, който може да валидира query execution contract-a без външна инфраструктура. Вторият е да има live тестове за реални backend-и, когато съответните targets и environment flags са налични.

Таблица 22. Каталог на integration и live тестовете

Файл	Режим на изпълнение	Какво демонстрира
test_mockdb.cpp	винаги наличен integration baseline	query composition, parameter binding, prepared queries и CRUD сценарии без външен backend
test_sqlite.cpp	локален integration тест	реален релационен execution path с capability asserts и hydration сценарии
test_postgresql_live.cpp	условен live тест	реална връзка, table lifecycle и CRUD изпълнение върху PostgreSQL
test_mysql_live.cpp	условен live тест	реален MySQL driver path, insert/update/delete и select filtering
test_mongodb_live.cpp	условен live тест	изпълнение на document-oriented сценарии срещу жив backend
test_redis_live.cpp	условен live тест	key-value и hash command materialization върху Redis среда
test_cassandra_live.cpp	условен live тест	wide-column execution и backend-specific CQL сценарии
test_neo4j_live.cpp	условен live тест	graph query execution през контролирана Docker среда

Този каталог е важен, защото показва, че repository-backed доказателството е разслоено. Не всички backend-и имат еднакъв operational cost за live verification, но хранилището съдържа ясна стратегия как и кога подобни тестове се активират.

11.4. Capability доказателства по backend семейства

Една от най-съществените идеи на библиотеката е, че connector-ите не трябва да претендират за повече възможности, отколкото реално предоставят. Затова capability доказателствата са разпределени между unit tests и integration tests, а не само между документационни твърдения.

Таблица 23. Примери за capability доказателства в тестовия слой

Backend	Основен тестов файл	Какво се доказва
SQLite	test_sqlite. cpp	налични supports_joins и supports_transactions в реален integration контекст
PostgreSQL	test_postgre sql_connecto r.cpp	налични joins, transactions и aggregation capability tags
MySQL	test_mysql_c onnector.cpp	налични joins и transactions, но не и aggregation capability
MongoDB	test_mongodb _connector.c pp	отсъствие на SQL-oriented capability tags и коректно filter rendering поведение
Redis	test_redis_c onnector.cpp	отсъствие на joins/transactions/aggregation и key-value materialization semantics
Neo4j	test_neo4j_c onnector.cpp	налични transactions и graph-specific execution focus
Cassandra	test_cassand ra_connector .cpp	ограничен capability профил и wide-column contract поведение

11.5. CMake registration като част от verification дизайна

Build системата не е неутрален фон, а активен компонент на верификацията. Това се вижда ясно от tests/unit/CMakeLists.txt и tests/integration/CMakeLists.txt. В unit слоя всички core тестове са обединени в един изпълним target и се регистрират чрез gtest_discover_tests. В integration слоя част от тестовете са винаги налични, а live тестовете се активират само когато съответният target и feature flag са налице.

```

1 add_executable(test_unit
2     test_types.cpp
3     test_property.cpp
4     test_relationship.cpp
5     test_query.cpp
6     test_mysql_connector.cpp
7     test_postgresql_connector.cpp

```

```

8     test_mongodb_connector.cpp
9     test_redis_connector.cpp
10    test_cassandra_connector.cpp
11    test_neo4j_connector.cpp
12    test_thread_safety.cpp
13    test_wire_protocol.cpp
14    test_cpp26_reflection.cpp
15    test_schema_migration.cpp
16 )
17
18 gtest_discover_tests(test_unit)

```

Listing 12: Извадка от unit test registration-a

Listing 12 е кратък, но много показателен: той описва unit verification слоя като систематичен каталог, а не като спорадично добавени тестове.

```

1 add_executable(test_integration
2     test_mockdb.cpp
3 )
4
5 gtest_discover_tests(test_integration)
6
7 if(TARGET orm::postgresql_live AND ORM_POSTGRESQL_LIVE_ENABLED)
8     add_executable(test_postgresql_live test_postgresql_live.cpp)
9     add_test(NAME PostgreSQLLive COMMAND test_postgresql_live)
10 endif()
11
12 if(TARGET orm::mysql_live AND ORM_MYSQL_LIVE_ENABLED)
13     add_executable(test_mysql_live test_mysql_live.cpp)
14     add_test(NAME MySQLLive COMMAND test_mysql_live)
15 endif()

```

Listing 13: Извадка от conditional live test registration-a

Listing 13 показва, че integration/live verification-ът е преднамерено conditional. Това е архитектурно разумно решение, защото не всеки backend е винаги наличен във всяка среда, но framework-ът за регистрация остава формално определен.

11.6. Изводи от приложението

Настоящото приложение показва, че верификацията на библиотеката има собствена вътрешна архитектура. Тя включва каталог на unit доказателства, baseline integration execution path, условни live сценарии и build-level механизъм за регистрация и активиране на тестовете. Това е важна част от изследователската стойност на разработката,

защото прави възможно твърденията от основния текст да бъдат свързани с конкретни, проверими артефакти.

Следователно дипломната работа не просто описва `library design`, а документира система, чиито основни тези могат да бъдат проследени до `source headers`, `CMake registration` и изпълними тестове. Именно тази проследимост е критична за оценяването на `Compile-time Object-Relational Mapping` библиотеката като завършена инженерна и академична разработка.

XII. АТЛАС НА CAPABILITY ПРОФИЛИТЕ И ЗРЕЛОСТТА НА BACKEND-ИТЕ

Настоящото приложение допълва основния анализ от глави VI и VII, като представя capability механизма и зрелостта на backend-ите в по-синтезиран вид. Ако предходното приложение систематизира тестовата инфраструктура, тук фокусът пада върху това как различните backend-и се позиционират спрямо общия connector contract.

Този тип синтез е полезен по две причини. Първо, той позволява да се разгледа библиотеката като координирана система от capability профили, а не просто като набор от отделни адаптери. Второ, дава възможност да се формулира по-ясно какво означава един backend да бъде минимално поддържан, operationally useful или архитектурно зрял.

12.1. Capability механизмът като формален слой

Файлът `lib/include/ORM/connector/capabilities.hpp` дефинира capability слоя чрез първичния шаблон `connector_trait<DB>` и чрез набора от capability tag структури в `namespace cap`. В разглежданата версия на библиотеката този набор включва `supports_joins`, `supports_transactions`, `supports_aggregation`, `supports_embedding`, `supports_upsert` и `supports_bulk_insert`.

Същественото е, че capability-ите не се пазят в runtime registry и не се описват като текстови метаданни. Те се кодират на ниво типове чрез присъствието или отсъствието на nested aliases в специализацията на `connector_trait<DB>`. Това означава, че допустимостта на дадена операция може да бъде използвана като compile-time условие в query API-то и в помощните концепти.

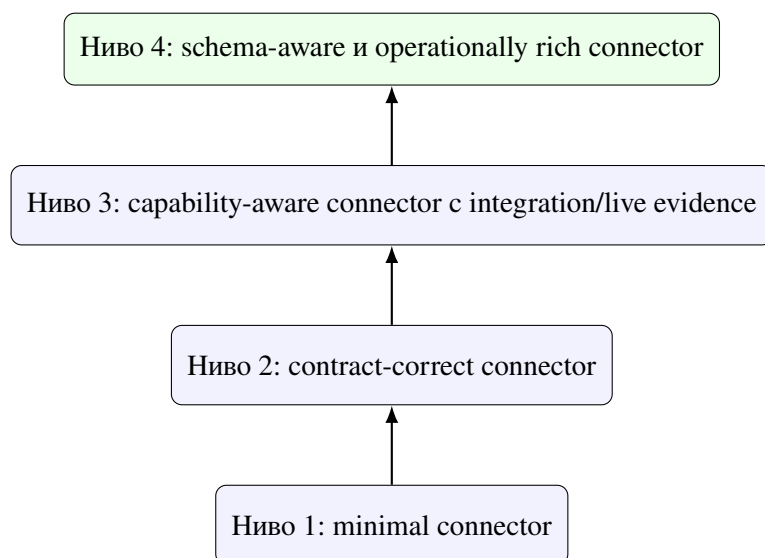
Таблица 24. Смисъл на capability tag-овете в разглежданата архитектура

Capability tag	Архитектурен смисъл
supports_joins	backend-ът може да материализира join-подобна композиция между логически entity описания
supports_transactions	backend-ът предлага достатъчна transactional semantics или API hooks за transaction helpers
supports_aggregation	backend-ът поддържа агрегиращи операции в рамките на съответния query модел
supports_embedding	backend-ът допуска document-oriented или nested object materialization стратегии
supports_upsert	backend-ът позволява едностъпково обновяване/вмъкване в подходяща форма
supports_bulk_insert	backend-ът разполага с ефективен път за batch insert сценарии

Таблица 24 е важна, защото показва, че capability слойът не е декоративен. Всеки tag трябва да има конкретно архитектурно значение, а не да бъде само описателен етикет.

12.2. Зрелост на конектора като стълбица, а не като двоична оценка

В практиката един connector рядко преминава директно от “липсва” към “напълно готов”. По-точно е да се мисли за него като за стълбица от нива на зрелост.



Фигура 19. Стълбица на зрелостта за backend connector-ите

Стълбицата от Фигура 19 позволява по-прецизна оценка на backend интеграциите:

1. **Minimal connector** — backend-ът има специализация на `connector_trait<DB>` и може поне да участва във формален `compile-time contract`.
2. **Contract-correct connector** — налични са `is_connector` доказателства и `unit` тестове за `capability` честност и базова `materialization` логика.
3. **Capability-aware connector с execution evidence** — налице са `integration` или `live` тестове, които показват, че `capability` профилът не е само декларативен.
4. **Schema-aware и operationally rich connector** — backend-ът може да участва в `migration`, `thread-safety`, `prepared-query` и по-богати `production-oriented` сценарии.

12.3. Capability профили по backend семейства

Синтезът на `unit` и `integration` тестовете позволява да се оформи компактна `capability` матрица за основните backend-и.

Таблица 25. Capability профили на основните backend-и според наличните tests и connector specializations

Backend	Joins	Transaction	Aggregation	Upsert	Bulk insert	Кратък коментар
SQLite	present	present	present	limited	limited	най-близкият пълен релационен reference path в хранилището
PostgreSQL	present	present	present	specific	possible	силен релационен contract с live execution evidence
MySQL	present	present	absent in current tests	specific	possible	operational path е наличен, но capability профилът е по-консервативен
MongoDB	absent	absent	absent	analogue	not central	силен document model, но не и SQL capability профил
Redis	absent	absent	absent	analogue	limited	key-value semantics с минималистичен contract профил
Neo4j		present	absent	not central	not central	graph backend със transaction focus, не със SQL-style aggregation
Cassandra	absent	present	absent	not central	possible	wide-column backend с ограничен, но формално дефиниран capability набор

Таблица 25 не трябва да се чете като универсална истина за самите бази данни. Тя описва конкретната архитектурна позиция на библиотеката спрямо тези backend-и в разглежданата версия на хранилището.

12.4. Карта на зрелостта по налична тестова опора

Сарабилити профилът сам по себе си не е достатъчен. От гледна точка на инженерната надеждност е важно дали за даден backend има само contract-level unit тестове, локални integration сценарии или и live execution доказателства.

Таблица 26. Backend maturity карта според наличната доказателствена опора

Backend	Ниво на зрелост	Основна доказателствена опора	Интерпретация
MockDB	ниво 3	test_mockdb.cp p	служи като backend-independent execution baseline за query contract-a
SQLite	ниво 3 към 4	test_sqlite.cp p	локален реален execution path с богата query evidence и capability asserts
PostgreSQL	ниво 3	test_postgresq l_connector.cp p, test_postgre sql_live.cpp	силен contract + conditional live backend проверка
MySQL	ниво 3	test_mysql_c onconnector.cpp, test_mysql_liv e.cpp	реален operational path с по-консервативен capability набор
MongoDB	ниво 3	test_mongodb_c onconnector.cpp, test_mongodb_l ive.cpp	document-oriented execution evidence без SQL-style capabilities
Redis	ниво 3	test_redis_c onconnector.cpp, test_redis_liv e.cpp	key-value execution evidence с ограничен, но честен contract профил
Neo4j	ниво 3	test_neo4j_c onconnector.cpp, test_neo4j_liv e.cpp	graph execution path с Docker-gated live verification
Cassandra	ниво 3	test_cassand ra_connector.c pp, test_cassa ndra_live.cpp	wide-column path с фокус върху формална пригодност и live coverage

12.5. Какво показва capability atlas-ът за архитектурата

От capability atlas-а следват три практически извода. Първо, библиотеката не се опитва да наложи изкуствена еднородност между backend-ите. Напротив, тя прави различията експлицитни чрез capability tags и contract-level tests. Второ, релационните backend-и естествено концентрират по-богат capability профил, докато документните, графовите и key-value backend-и се оценяват през други, по-подходящи критерии. Трето, зрелостта на конектора трябва да се измерва не само по броя на поддържаните операции, а и по наличието на реална тестова опора.

Тези изводи са важни и от изследователска гледна точка. Те показват, че compile-time ORM архитектурата е жизнеспособна именно защото не заличава различията между storage моделите. Вместо това тя ги формализира и прави управляеми на ниво типове, capability профили и verification стратегии.

12.6. Извод

Настоящото приложение завършва синтеза между connector contract-a и empirical evidence слоя. Ако основният текст показва как библиотеката работи, capability atlas-ът показва как тя структурира разнообразието от backend-и по дисциплиниран и проверим начин. Това е още една причина разработката да бъде разглеждана не само като програмна библиотека, а и като формално организирана инженерна система.